SE/EE 491 Project
Software Engineering-Electrical Engineering Departments
Dec 2025

Solar powered Smart Entry System with AI threat Detection (Bawaba 2030)

Students' Name & ID:

Abdulmoula, R. Rama - 4210026 (4210026@upm.edu.sa)

Alshel, L. Lama - 4110108 (4110108@upm.edu.sa)

Yousef, A. Aseel - 4210307 (4210307@upm.edu.sa)

Under the Supervision of:

Dr. Ahsan Rahman (a.rahman@upm.edu.sa) / EE

Dr. Hazrina Sofian (h.sofian@upm.edu.sa) / SE

University of Prince Mugrin

Year 2025 - 2026 Fall

ANTI-PLAGIARISM DECLARATION

This is to declare that the above publication produced under the supervision of Dr. Hazrina Sofian and Dr. Ahsan Rahman having the title "Solar powered Smart Entry System with AI threat Detection" is the sole contribution of the authors and no part has been reproduced illegally, which can be considered as plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: 12/12/2025

Author(s):

Name: Rama Mohamad Signature: RAMA

Name: Lama Alshel Signature: LAMA

Name: Aseel Yousef Signature: ASEEL

I. ACKNOWLEDGMENT

First and foremost, we express our deepest gratitude to Allah Almighty for granting us the health, knowledge, and perseverance needed throughout this project journey. We would also like to extend our sincere appreciation to Dr. Hazrina Sofian and Dr. Ahsan Rahman for their continuous guidance and support, which greatly enriched our learning experience. Furthermore, we are grateful to the Software Engineering Department, Electrical Engineering Department, and the College of Computer Science for their valuable support.

II. ABSTRACT

This report presents the analysis, design, and proof-of-concept validation of a Solar-Powered Smart Entry System with AI Threat Detection (Bawaba 2030) for a single vehicle gate at the University of Prince Mugrin (UPM) – Male Campus, Madinah, Saudi Arabia. The work targets slow manual checks, inconsistent logging, and security blind spots, while recognizing that high-cost biometric solutions are often impractical. The objectives are to (1) analyze the limitations of manual and automated gate operations in security and entry speed, (2) design a dynamic QR-based contactless authentication workflow with real-time, auditable logging, (3) evaluate operation and security under varied access scenarios, and (4) support continuous functionality using a solar panel and battery system. Methods followed the Digital Project Management Guideline for planning and governance, using Jira Kanban (To-Do → In-Progress → Under Review → Done) and GitHub version control for code and documentation. Requirements were specified using ISO/IEC/IEEE 29148:2018 with uniquely identified FR/NFR/HR items, and the architecture was selected via Kepner-Tregoe Decision Analysis, resulting in a Hybrid Layered with 3-Tier design. A Django backend simulation, tested through Postman, demonstrated QR token issuance and validation, permit/deny decisions, and fail-closed behavior; log schemas and rule-based anomaly flags (e.g., repeated denials and after-hours attempts) were defined, and a solar/battery sizing worksheet was produced from stated assumptions. Findings indicate that the proposed design supports rapid access decisions, complete traceability through structured logs, and clear operational separation between the web client, backend/API, gate service, and Arduino actuator interface. The discussion highlights improved security for the campus environment, with scalability to multiple gates enabled by the chosen architecture

Keywords*: Smart Gate System, Dynamic QR Code Authentication, AI Threat Detection, Access Control, Solar-Powered Systems, Log Management, Three-Tier Architecture, Campus Security.*

III. TABLE OF CONTENTS:

IV. LIST OF FIGURS

VI. LIST OF TABLES

V. ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
DGA	Digital Government Authority
EE	Electrical Engineering
FR	Functional Requirement
GMT+3	Gulf Standard Time (Saudi Time Zone)
HR	Hardware Requirement
HTTPS	Hypertext Transfer Protocol Secure
KT-DA	Kepner-Tregoe Decision Analysis
MVP	Minimum Viable Product
NCA	National Cybersecurity Authority
NFR	Non-Functional Requirement
OTT	One-Time Token
QR	Quick Response (Code)
REST	Representational State Transfer
SE	Software Engineering
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

CHAPTER 1: INTRODUCTION AND BACKGROUND

1.1 Project Overview

Modern universities require efficient and secure gate access systems to manage traffic and improve campus safety. Traditional gate management relies on manual identity verification by security personnel, which often leads to delays, human error, and security gaps.

This project presents a Solar Powered Smart Entry system with AI-Based Threat Detection, providing automated, and contactless access control. Authorized users gain entry using QR codes, offering a practical and cost-effective alternative to facial recognition systems, which are expensive and unreliable in outdoor environments.

The system is powered by solar energy, ensuring sustainable operation and reduced dependence on the main power grid. An Al based threat detection module monitors unauthorized activities and maintains automated entry logs and enhances overall security.

1.2 Vision of the project

To realize a scalable, secure, and energy-efficient smart entry system for university and institutional campuses that, auditable, and user-friendly gate access—contributing to Saudi Vision 2030 goals for digital transformation and smart infrastructure.

1.3 Mission of the project

To design, implement, and pilot a solar-powered smart gate system that provides rapid, contactless, and secure access through dynamic QR code authentication. The system integrates a local gate PC and an Arduino-based actuator, records all access attempts to ensure auditability and traceability, and incorporates AI-based threat detection and monitoring mechanisms, with the objective of achieving high availability during defined operating periods.

1.4 Project Plan

1.4.1 Deliverables

- Functional simulations representing wiring diagrams, flowcharts, and feedback loops.
- 3D model demonstrating system components layout.
- Requirements documentation (FR, NFR, constraints).
- System Architecture and UML Diagrams.
- UI/UX Wireframes and interaction flows.
- Integrated SE–EE design plan (communication protocol, command mapping).
- Capstone I Final Report.

1.4.2 Estimated Budget

Component	Quantity	Unit Cost (SAR)	Total Cost (SAR)
Arduino Uno	1	70	70
DC Motor 12V	1	150	150
Battery 12V	1	99	99
QR Scanner	1	200	200
Raspberry Pi	1	299	299
Relay Module	1	76	76
Charge Controller	1	32	32
Total	8	926	926

Table 1. Hardware cost estimation for the Smart Gate System.

1.4.3 Gantt chart

The project timeline for **Capstone I** follows the Software Development Life Cycle (SDLC) using the proposed Hybrid Model, which integrates both Waterfall and Agile approaches. This model ensures that the foundational phases—such as problem identification, requirements engineering, and system design—are completed in a structured and sequential manner (Waterfall), while allowing iterative refinement and continuous improvement during documentation and review stages (Agile).

Additionally, the timeline is designed to align with the collaborative nature of this interdisciplinary project, ensuring smooth coordination between both Software Engineering (SE) and Electrical Engineering (EE) tracks. Shared phases—such as requirements, architecture decisions, and integration planning—are synchronized, while discipline-specific tasks are distributed to match each team's responsibilities.

The planned progression of all major activities throughout Capstone I is summarized in Figure 1, which presents the detailed SDLC-based timeline covering project initiation, literature review, requirements analysis, SRS development, system design, UI/UX preparation, evaluation, and final documentation.

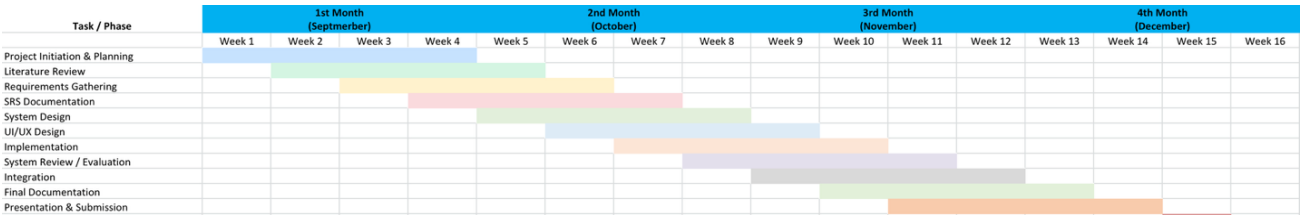


Figure 1. Gantt Chart outlines task and milestone.

1.5 Link to SE and EE Courses

1.5.1 Software Engineering Courses

1.Software Requirements Engineering (SE311):

- Requirements elicitation and stakeholder analysis.
- Developing and validating use cases.
- Defining functional and non-functional requirements (security, real-time access, usability).
- Ensuring requirements fit budget, environment, and technology constraints.

2. Software Architecture & Design (SE342):

- Designing a modular and scalable architecture suitable for real deployment.
- Mapping requirements to architectural views and component interactions.

3.Software Process and Modeling (SE323):

- Applying process models such as Waterfall, Agile, and Hybrid models used in Capstone.
- Using modeling techniques (UML: Use Case, Class, Sequence, Activity, Deployment).
- Selecting the Hybrid methodology based on project constraints (software + hardware integration).
- Ensuring the development process follows structured phases aligned with SE standards.

4. Software Project Management (SE464):

- Using Jira for task management, progress tracking, and Kanban workflow.
- Using GitHub for version control, documentation management, and traceability.
- Aligning project planning with Digital Project Management Guidelines (DGA).

5.Ethics and Professionalism (SE372):

- Applying the Software Engineering Code of Ethics in system design.
- Ensuring privacy, transparency, confidentiality, and responsible data handling.
- Maintaining professionalism, fair credit distribution, and ethical decision-making.
- Designing a system that protects users and enhances safety for the university.

1.5.2 Electrical Engineering Courses

1. **Electric Circuits I (EE201):**

- For understanding the design of the circuit and protection system.
- Calculating the parameters for safe and efficient operation
- Essential for supplying the DC motor of the gate and microcontroller properly.

2. **Introduction to Modern Power System (EE356):**

- Essential for designing a stable power supply for motor and electronics.
- Providing knowledge on protection devices.
- Calculating power requirements for entire system.

3. **Control system Analysis (EE461):**

- Ensuring smooth and precise gate system.
- Allowing integration of feedback from sensors for automatic control.
- Helping in controlling motor speed, acceleration, and stopping accurately.

4. **Electric Machines (EE351):**

- Selecting and understanding the DC motor for the gate.
- Calculating to choose the right motor.
- Understanding motor behavior and interfacing with relay control for forward and reverse motion.

1.6 Problem Statement

Many organizations' access systems still operate without fully automated gate control and without intelligent log management, so access events are not consistently captured or correlated with user identity. Due to the growth of new hacking tools and the evolving capability of attackers' systems, such gaps make it easier to breach the gate workflow, resulting in unauthorized access without alerting the security guard [3]. Due to the heavy reliance of traditional automatic gates on the main power grid, the gate becomes vulnerable during power outages, operating costs increase, and the system cannot guarantee continuous secure operation. These problems reduce overall system reliability, increase maintenance burden, and create windows where the gate may stay open or unmonitored, which motivates the need for an automated, logged, and power-resilient gate solution.

1.7 Objectives

1. To analyze the limitations of automated and manual gate operations particularly in security, entry speed, and assess their overall impact on campus operations.

2. To design and develop an automated campus gate system that uses dynamic QR-based contactless authentication and a threat-detection algorithm, while enhancing security through access logging that enables real-time traceability.

3. To evaluate the operation and security of the smart campus gate system that utilizes dynamic QR-based contactless authentication and AI-powered threat detection, ensuring secure access control and real-time logging under various access scenarios.

4. To design an automatic gate that minimizes energy consumption by utilizing solar energy as a sustainable power source.

5. To provide a prototype that demonstrates smooth operation, reliable access control, and continuous functionally even during power outages.

6. To evaluate the operation and implement the operation of motorized gate system controlled by Arduino and sensors, powered by a solar panel as alternative renewable energy source.

For more detailed information please refer to (Appendix G).

1.8 Project Scope

This project focuses on analyzing the requirements of a smart campus gate system and proposing practical solutions to the identified problems, which are later translated into clear system requirements across different system entities. The project is implemented in the Kingdom of Saudi Arabia, Madinah, at the University of Prince Mugrin (UPM) – Male Campus, and targets a single vehicle access gate within the campus.

The scope addresses limitations in existing gate access systems, including the lack of fully automated gate control, weak smart log management, evolving attacker capabilities, heavy reliance on the main power grid, and reduced system reliability. These issues create security gaps and operational inefficiencies in traditional gate operations.

The proposed system introduces a fully automated gate control mechanism supported by digital logging and event tracking, ensuring that every access event is captured and accurately linked to the correct user identity. The system relies on dynamic QR code–based access verification, which, according to the comparative analysis in Chapter 2, outperforms facial recognition solutions in terms of cost, processing speed, and feasibility for the university environment. Automated digital logs enhance anomaly detection and provide clear visibility into system activity.

To counter modern security threats, the project scope includes the integration of encrypted, time-limited QR codes combined with AI-based threat detection mechanisms. The system detects abnormal access behaviors, such as repeated access attempts within a short time window or access attempts outside normal operating hours, and flags them as suspicious. AI-driven log analysis improves anomaly detection accuracy and strengthens overall system resilience without relying on manual observation or static credentials.

Additionally, the scope incorporates the use of a solar-powered energy system with battery storage to ensure continuous gate operation during power outages while reducing long-term operational costs and dependence on the main electrical grid. The project also enhances reliability through automated access validation, dynamic logging, and real-time alerts, minimizing scenarios where the gate may remain open or unmonitored. The scope of this project is limited to analysis, design, and simulation, without full production deployment.

Out-of-Scope Items

To prevent scope expansion, the following elements are explicitly excluded:

Payment systems, parking management, visitor access passes, or integrations with external third-party systems (e.g., national identity platforms or external HR systems).

CHAPTER 2: LITERATURE REVIEW AND ANALYSIS

2.1 Overview

This chapter provides a review of relevant literature, including research papers and the existing systems. The chapter sets the foundation for identifying gaps in current systems and demonstrates the need for the proposed solution. As well as reviewing the possible solutions.

2.2 Key Studies Analysis

Three key studies were selected based on their relevance and contribution to the proposed Automated Gate Access System. These papers provide strong theoretical and technical foundations for improving system performance, reliability, and security.

Reference [3] discusses how log management and automated threat detection can enhance the overall security of smart systems. It highlights the role of real-time logging in detecting suspicious activities and ensuring compliance with data protection standards. The ideas from this paper are applicable to our project in improving event threat detection to improve the security for gate access systems. The emphasis on automation also aligns with the Saudi Vision 2030 initiative for smart campus development.

Reference [13] presents an Intelligent Automated Gate System (IAGS) using QR codes for access control. The design integrates both hardware (Arduino, PIR sensor, servo motor, LED, buzzer) and software (VB.NET) components. Although the system achieved 99% accuracy, it faced challenges due to lighting conditions and internet speed affecting QR readability. The proposed project aims to enhance this design by improving QR detection accuracy, supporting offline access verification, and adding notification features for real-time access updates to improve efficiency and reliability.

Reference [9] directly inspired the project concept. It introduces a Smart Campus Gate System using QR Code, which focuses on automating campus entry and exit management. The paper demonstrates how QR-based control can reduce waiting time, increase security, and simplify gate operations. Building upon this concept, our project seeks to make the system faster, cheaper, and more accurate through improved algorithms, efficient hardware–software communication, and real-time monitoring features suitable for university environments.

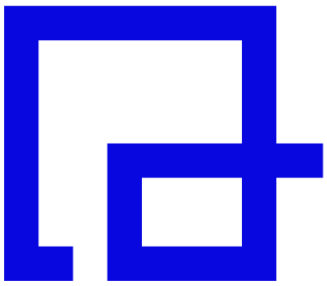
See **Appendix F** for the summary of some studies.

2.3 Similar Systems Comparison

2.3.1 QrTrac's [20]

QR Code gate automation System is a reference for our project, as it also automated gate using QR code. It generates codes with embedded ticket and user data, validates them via API.

Limitations:



1.Security Risks

Physical tampering or copying QR codes is possible.

Network/Server Dependence

Gate operation relies on API validation; outages can prevent access.

2.Time and Data Constraints

Precise time synchronization is needed; large codes may slow scanning.

3.Limited use cases

Best suited for controlled access since it doesn't handle offline users.

2.3.2 NineSmart Smart Access System [21]

NineSmart provides a cloud-based smart access control solution that utilizes dynamic QR codes to manage secure entry to buildings and gated areas. The system allows authorized users, employees, and visitors to access doors or gates by scanning a dynamically generated QR code using their mobile devices. Each QR code is time-based and continuously updated, which enhances security and prevents code reuse or unauthorized sharing.

The NineSmart Smart Access platform offers centralized management through an administrative dashboard, where access permissions can be granted, modified, or revoked in real time. It also supports temporary QR codes for visitors, making it suitable for office buildings, educational institutions, and residential facilities. By eliminating the need for physical access cards or manual verification, the system improves operational efficiency while maintaining a high level of security and traceability through access logs.

This solution demonstrates a real-world implementation of dynamic QR-based gate and door access control, which closely aligns with the concept proposed in our project.

Limitations:

1.Despite its advantages, the NineSmart Smart Access system has several limitations. First, the system relies heavily on smartphone availability and internet connectivity, meaning users must have a mobile device with sufficient battery and network access to generate and display the dynamic QR code. This may cause access issues in cases of poor connectivity or device failure.

2.Second, QR code–based access systems are less secure than biometric-based solutions in scenarios where identity verification is critical, as QR codes can still be captured or misused if a device is compromised. Although dynamic QR codes reduce this risk, they do not fully eliminate the possibility of unauthorized access compared to facial recognition or fingerprint authentication.

3.Additionally, the system depends on camera or scanner accuracy at the gate, which may be affected by environmental factors such as poor lighting, screen brightness, or damaged device displays. Finally, managing visitor access at very high traffic volumes may introduce delays, as each QR code must be individually scanned and verified.

2.4 Possible Solutions

In our search for a secure and fully contactless access method, we reviewed and compared two research papers that implement contactless entry techniques: one using QR codes [13] and the other using facial recognition [4]. To select the most suitable approach for our project, we evaluated both technologies based on a set of constraints relevant to our system requirements. We assigned a weight to each constraint to represent its importance in our project, ensuring a more accurate and meaningful comparison. As shown in Table (2)

Constraint	Weight (Σ = 1)	QR Code (0–5)	Facial Recognition (0–5)	Weighted Score QR	Weighted Score Facial	Explanation
Time to develop	0.15	5	2	0.75	0.3	Developing a QR based system takes less time since it uses simple libraries and minimal setup. Facial recognition requires more time for model training, dataset preparation, and hardware calibration.
Cost	0.20	4	1	0.8	0.2	QR systems are low cost (only need cameras and code generation), while facial recognition is expensive due to high-resolution cameras, servers, and AI software.
Scope	0.10	3	2	0.3	0.2	QR codes are easier to expand, new users just need new codes. Facial recognition requires adding and retraining face data, which is slower to scale.
Security	0.15	3	5	0.45	0.75	Facial recognition provides stronger identity verification because it's harder to forge a face than a code.
Maintenance	0.10	4	2	0.4	0.2	QR systems require minimal updates (mostly regenerating codes). Facial recognition needs ongoing updates to improve accuracy and handle lighting or camera changes.

Accuracy	0.10	4	4	0.4	0.4	Both perform well, but facial recognition can misread under poor lighting or occlusion, while QR may fail if printed or displayed poorly.
Hardware Requirements	0.10	5	2	0.5	0.2	QR requires only standard cameras or smartphones, while facial systems need advanced cameras and possibly GPU powered servers.
Privacy & Regulations	0.10	5	3	0.5	0.3	QR codes handle limited personal data, making them compliant and easy to deploy. Facial recognition involves biometric data and faces stricter privacy laws and ethical concerns.

Table 2. Comparison Matrix Between QR Code Access and Facial Recognition

For example, we assigned 15% to development time because speed is important but not the top priority. We assigned 20% to cost because it is one of the most influential factors in our final decision. We assigned 15% to security due to the sensitivity of our environment. We assigned 10% to privacy and regulations because privacy matters in Saudi Arabia, however it is not the biggest limitation at this stage.

Each technology was rated from 0 to 5, where 5 means excellent performance and 0 means poor performance. We then multiplied each rating by its weight to obtain the weighted score and calculated the total score for both methods.

The results were:

- Total Weighted Score (QR) = 4.1 / 5
- Total Weighted Score (Facial Recognition) = 2.55 / 5

Therefore, since the QR method achieved a higher total score and is faster and more cost-effective to implement, we decided to proceed with the QR-based access system for our project.

Explanation of Comparison Constraints

1. Time to Develop: How long it takes to build and complete the system.
2. Cost: The total money required to finish the project.
3. Scope: The size and complexity of the system, including features and tasks.
4. Security: The ability of the system to protect identity and prevent unauthorized access.
5. Maintenance: How easy it is to update or fix the system after deployment.
6. Accuracy: How reliably the system reads or identifies the input (QR code or face).
7. Hardware Requirements: The devices or equipment needed for the system to function.
8. Privacy & Regulations: How well the system protects personal data and complies with privacy laws.

CHAPTER 3: METHODOLOGY

3.1. Overview

Since our capstone project spans two fields (hardware and software), our teamwork in parallel and meets at planned checkpoints. Therefore, relying on a pure methodology is not suitable for our context.

3.2. Methodologies to Consider

For a pure Waterfall approach [1], advantages include: the number of resources needed is predefined and minimal, a sequential model is rather easy to implement, verification and validation in each phase reduce the chance of error, and the stages are easy to understand and well defined. However, disadvantages include going back to the previous stage during development is not possible, changes are hard to implement, client involvement and feedback are significantly less, and the error of one phase will impact the other stages. In our capstone context, this becomes more critical because Capstone 1 freezes Requirement Analysis to Design, so any new constraints discovered in Capstone 2 (during implementation or hardware integration) cannot be incorporated smoothly, making the second semester rigid and high-risk. As shown in Figure (2).

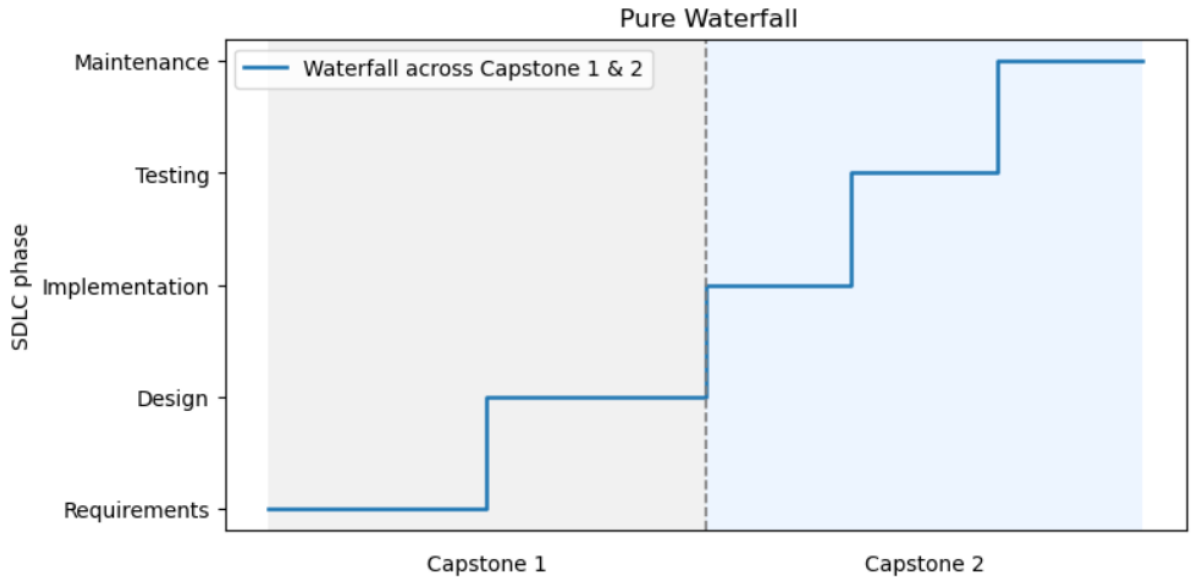


Figure 2. SDLC plot for the two capstones using a pure Waterfall model.

For a pure Agile approach [1], advantages include: Small increments in software/product delivery result in a satisfied and happy customer, more adaptive and change friendly, communication is very much valued, more transparent approach, value daily and constant human interaction. However, disadvantages include: Complicated when it comes to big teams or more teams working on one project, it gives scope creep, it makes it difficult to set a fixed budget and end date of the project and less predictive. Moreover, pure Agile assumes all SDLC activities (requirements, design, implementation, and testing) evolve together in short cycles, which clashes with our capstone structure that requires us to complete Requirements and Design in Capstone 1 and start the implementation and testing in Capstone 2. As shown in Figure (3).

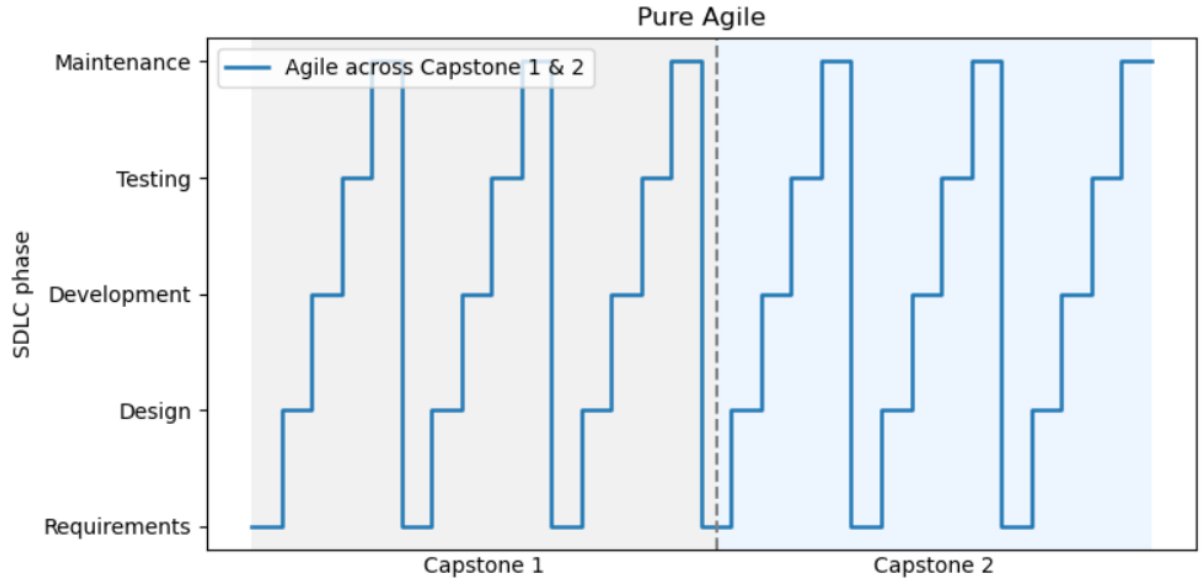


Figure 3. SDLC plot for the two capstones using a pure Agile model

3.3 Our Methodology

Based on our observations and literature review analysis, we will adopt a Hybrid process [2], because it better supports cross-team collaboration. Hybrid gives us four things we need in a hardware–software capstone: **compatibility** (it works with mixed project types), **clarity of responsibilities** (the project is mapped from start to finish with defined roles), **detailed planning** (we can meet academic milestones), and **flexibility** (we can still adjust when implementation reveals new constraints). Since Capstone 1 is structured analytically (SRS to Design) and Capstone 2 is for execution (Implementation and Testing), pure Waterfall would freeze decisions too early, while pure Agile would not match the staged academic deliverables. Therefore, we will set a controlled baseline design in Capstone 1, and in Capstone 2 we will implement iteratively with scoped design adjustments informed by implementation results. This Hybrid methodology gives us both predictability and adaptability. As shown in Figure (4).

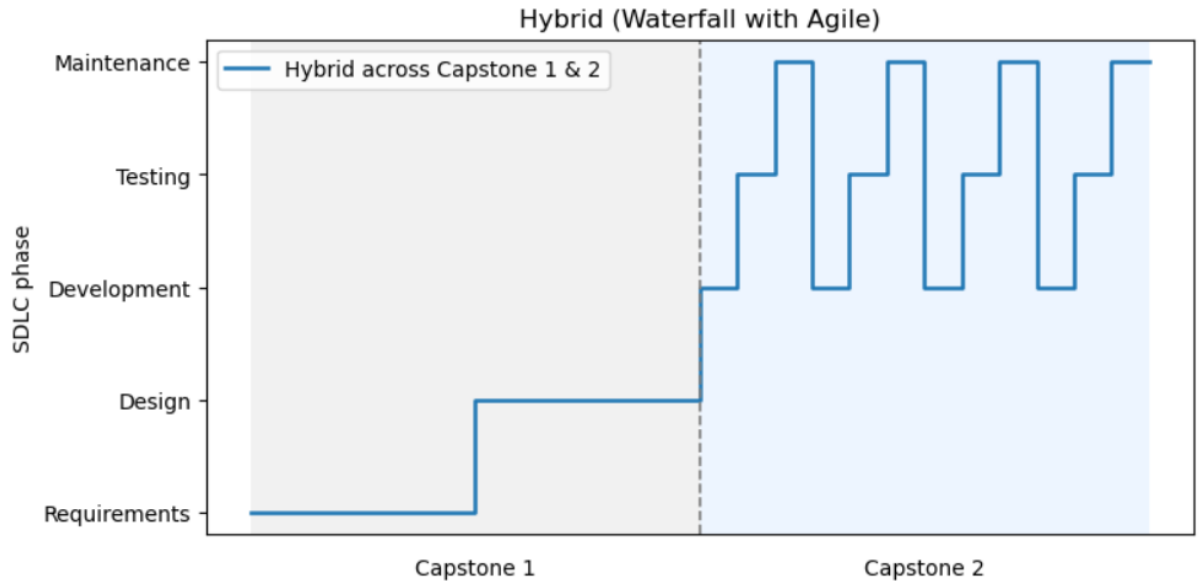


Figure 4. SDLC Plot for the two capstones using the proposed Hybrid (Waterfall & Agile) model

3.4 Team Roles and Responsibilities

We created a Hierarchical Task Analysis to show the main functional areas of our (Bawaba 2030) and to clarify how tasks are divided between the SE and EE team members. As shown in Figure (5), the system is organized into three major domains: the user application, the backend and core services, and the hardware components. Each domain is broken down into its key responsibilities such as account management, QR handling, application algorithms, database logic, power management, and motor control. This diagram helps visualize how the team collaborates to develop an integrated and multidisciplinary solution.

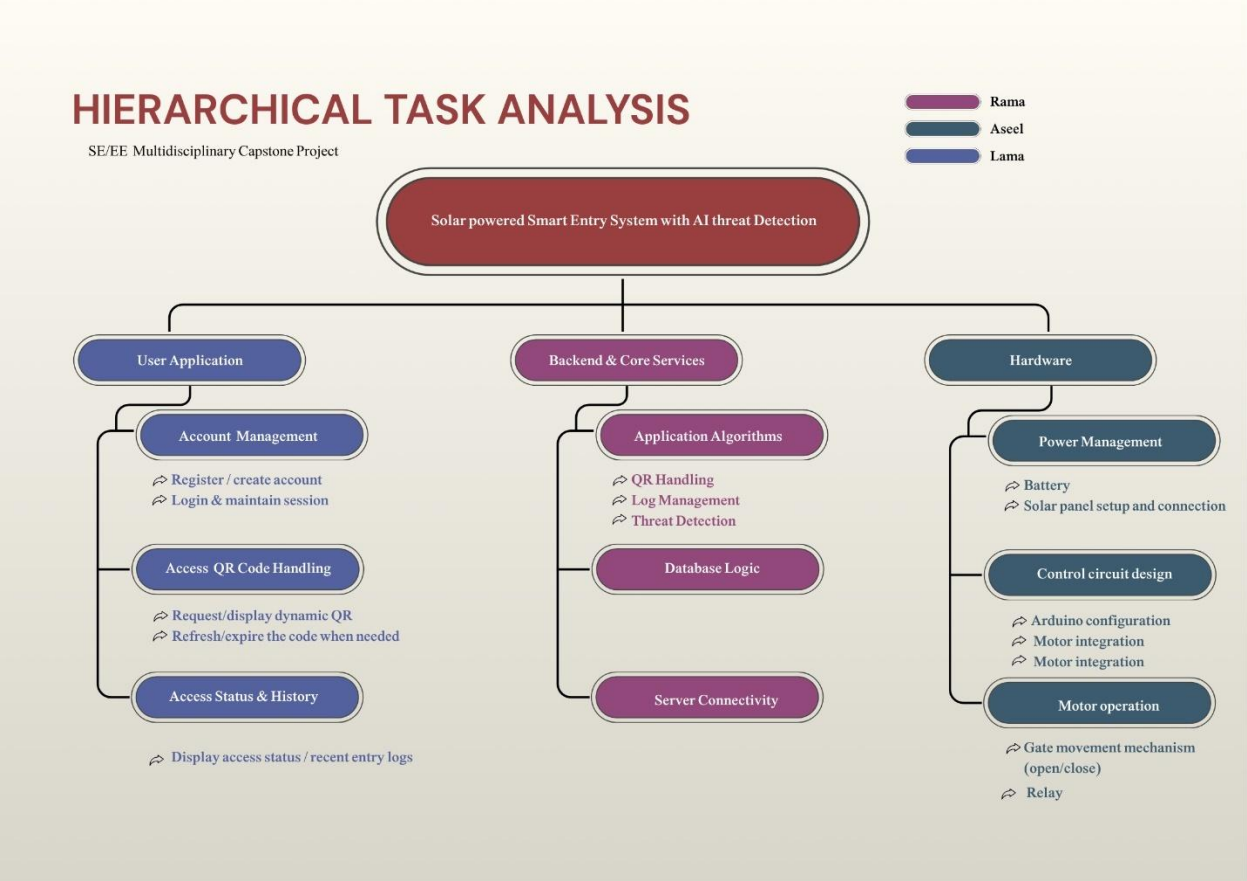


Figure 5. Hierarchical Task Analysis showing the main system components and the task distribution across the SE and EE team members

3.5 Project Management Tools and Techniques

In this part of the document, we followed the **Digital Project Management Guideline** [9] framework as the guiding method to plan for this project. We applied several of its recommended practices, including:

1. Applying Section 5.4.1 – Choosing an Appropriate Methodology

We selected the methodology that fits our project characteristics and constraints, following the guideline's criteria for planning and implementation.

2.Applying Section 5.4.4 – Tools and Technologies of Agile Development

In alignment with the guideline's recommended tools, we used Jira Kanban boards to manage our tasks, track progress, visualize workflow (To-Do → In-Progress → Done), and maintain team alignment as shown in figure (6). **Note**: It is worth noting that we added an additional column labeled "Under Review", even though this stage is not included in the original Digital Project Management Guideline. We introduced this step to better reflect our internal process, where completed tasks must be reviewed and verified by team members before being officially marked as done. This ensured higher quality control, clearer responsibility distribution, and more accurate tracking of deliverables. Please refer to (Appendix B) to access the Jira.

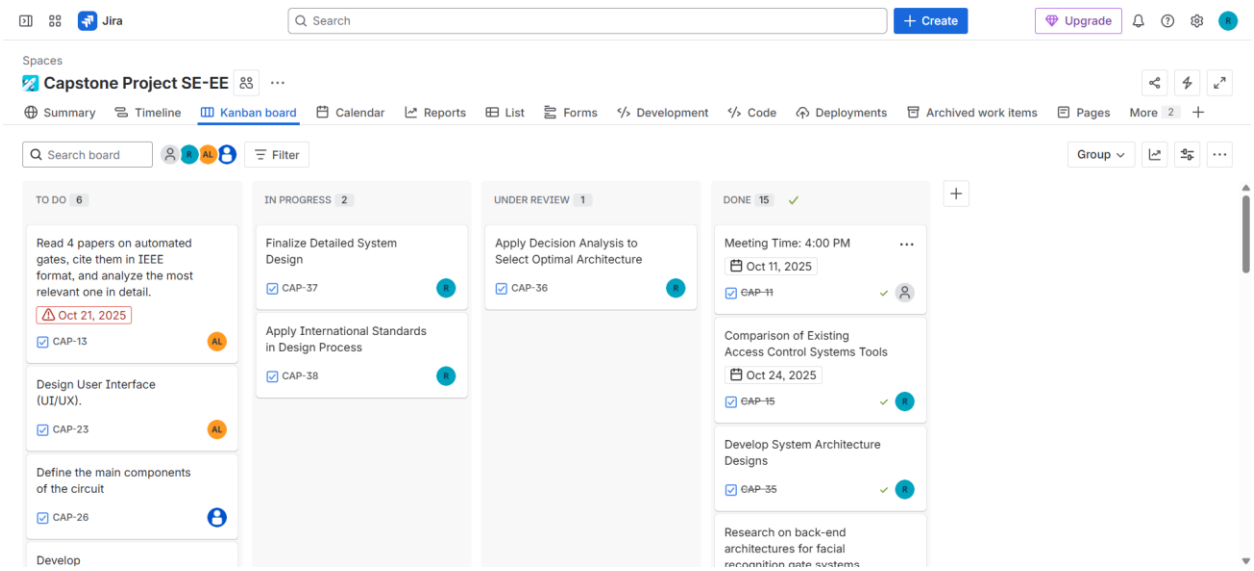


Figure 6. Jira using Kanban board

3.Applying Section 5.9.2 – Version Controlling

As recommended in the guideline, we used GitHub for version control not only for source code, but also for documentation, simulations, models, and diagrams to maintain traceability, transparency, and organized version history throughout the project. As shown in Figures (7) and (8). Please refer to (Appendix A) to access the project repository.

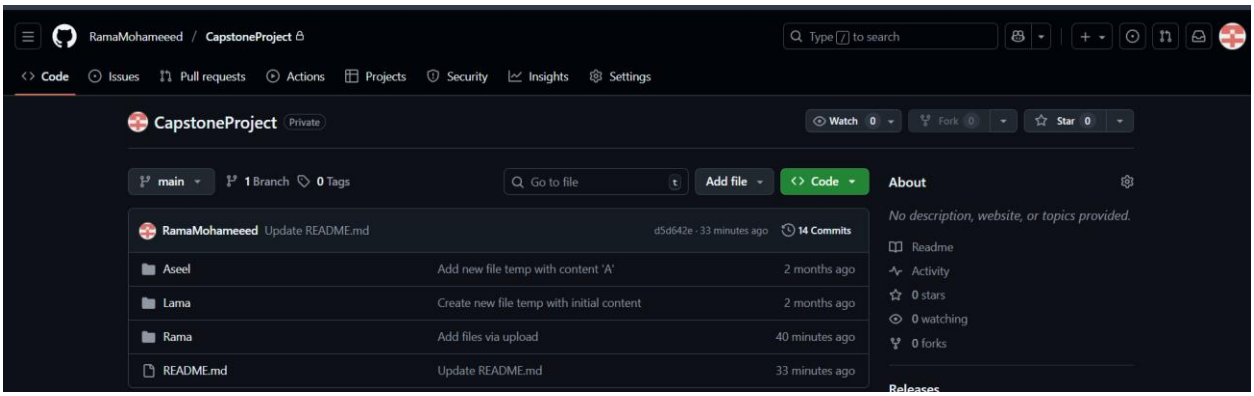


Figure 7. GitHub repository to store and organize all project materials

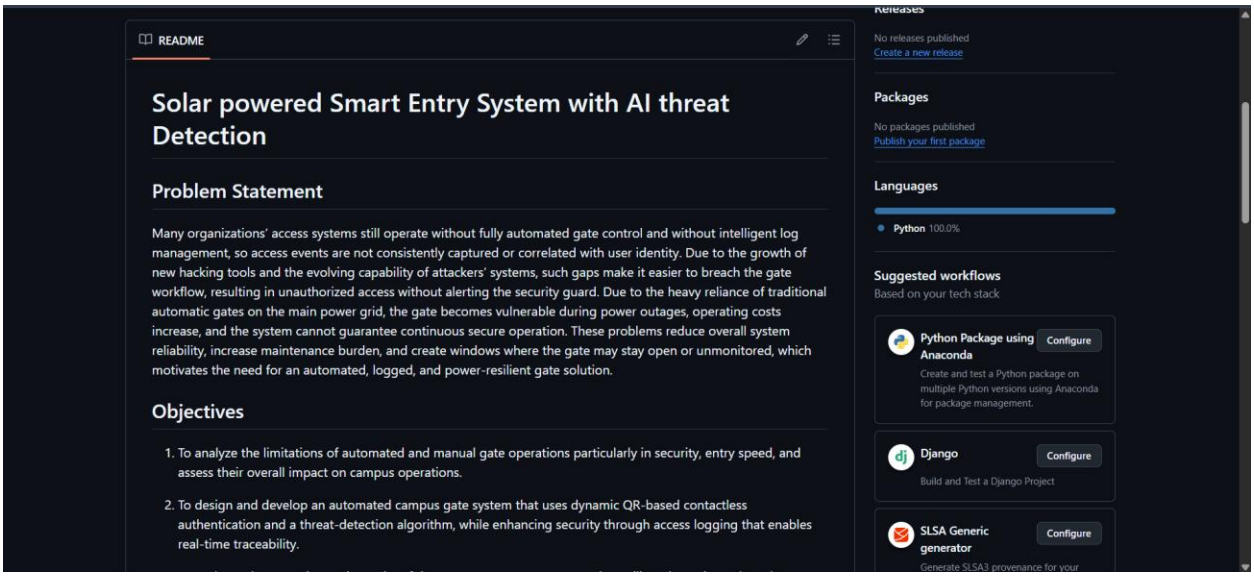


Figure 8. ReadMe file in GitHub

4.Applying Section 5.3.3 Investment in Technologies

We applied Jira Atlassian to manage tasks, as shown in Figures (9), (10), and (11). Organize team responsibilities, monitor progress, and maintain clear visibility of the workflow throughout the development process. Also, for cooperation and communication tools; we used a dedicated Microsoft Teams chat as shown in figure (12) where we held our weekly online meetings, shared updates, and coordinated decisions.

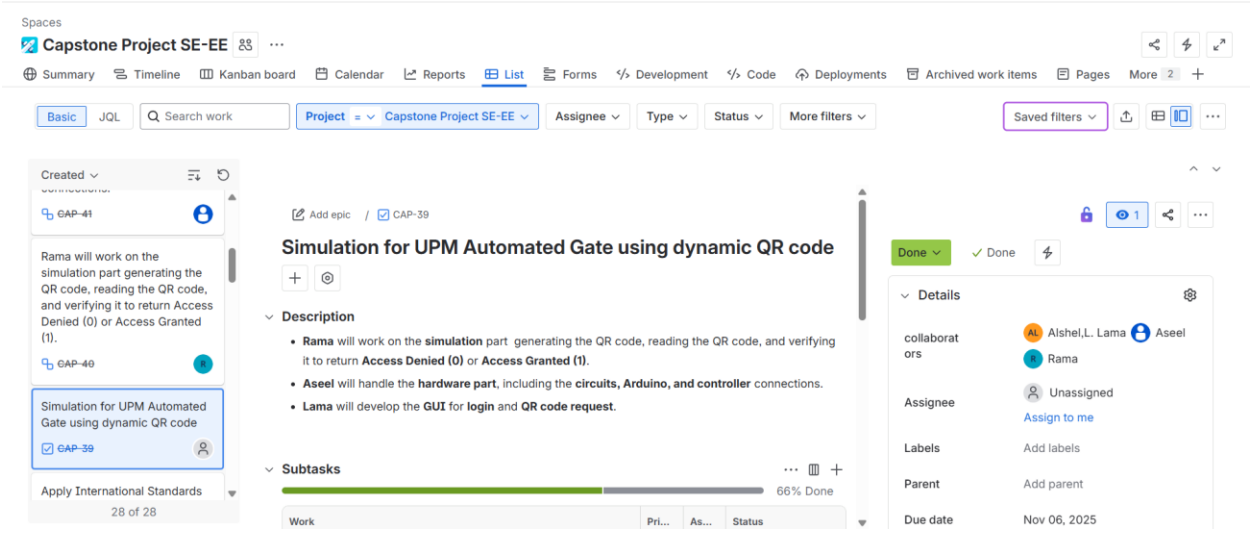


Figure 9. Jira task showing the simulation work for QR code generation and gate access handling.

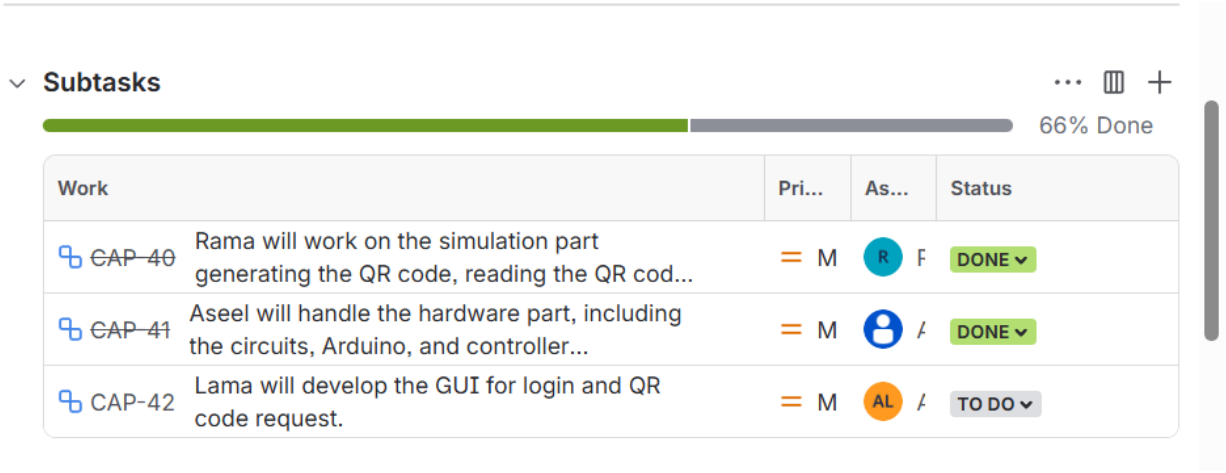


Figure 10.Jira subtasks distribution and progress (66% completed) among team members.

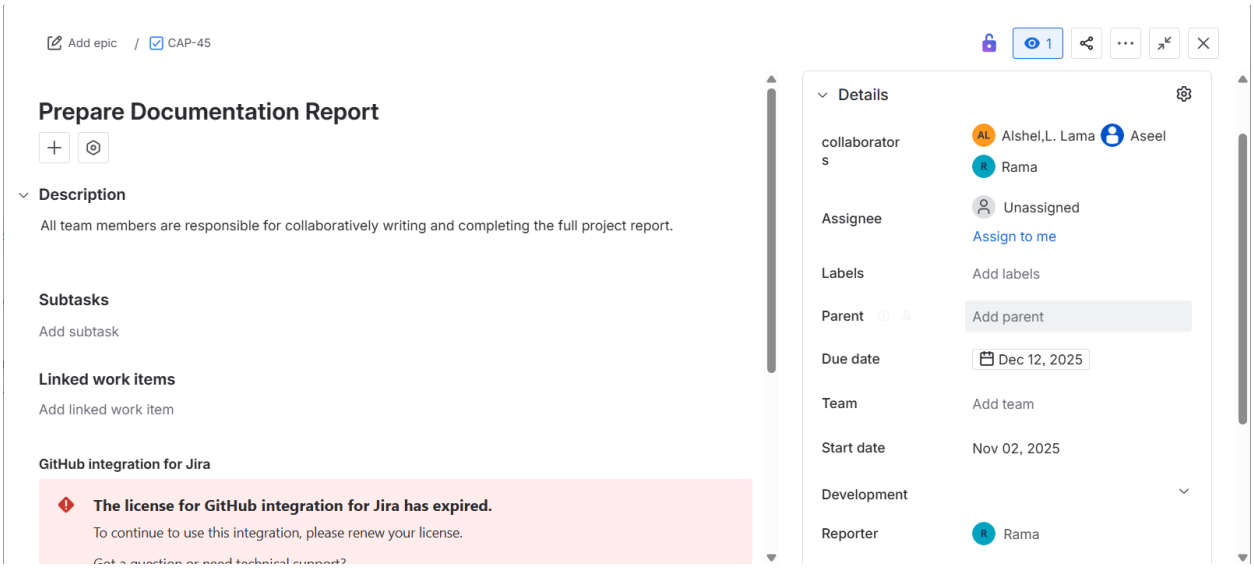


Figure 11. Jira task for preparing the full documentation report with team responsibilities.

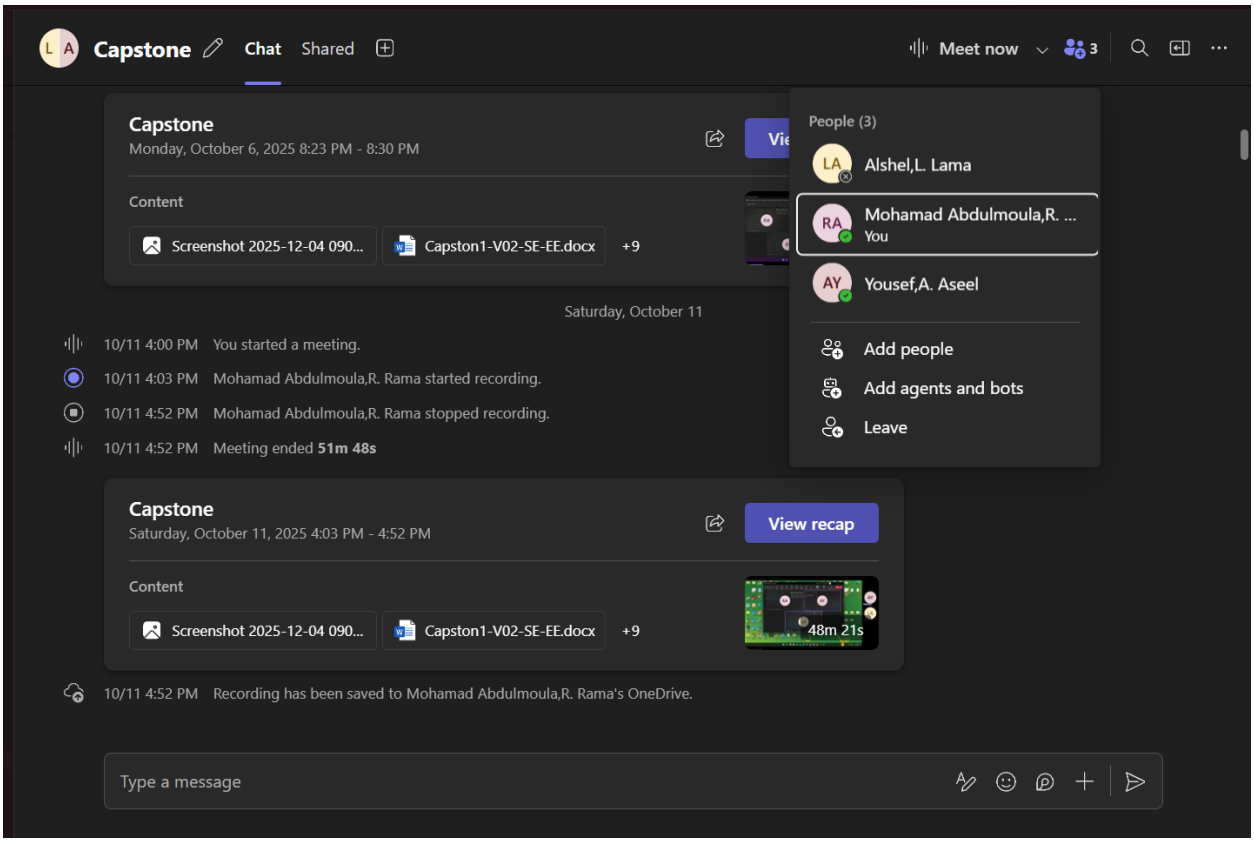


Figure 12. Microsoft Teams meeting showing team collaboration, recorded discussion, and shared project files.

Note that, in addition to the recommended tools, we maintained a WhatsApp group to enable fast, informal communication for quick clarifications and immediate discussions. Although this is not

required in the guidelines, we adopted it to improve coordination and responsiveness within the team (see Appendix E).

Regular weekly meetings were conducted with the project supervisors. The detailed meeting schedule is provided in the (Appendix D).

These practices helped us align with national digital project management standards and ensured that our workflow followed a structured, well-governed, and professional methodology.

3.6 Business Model Canvas

As shown in figure (13), we developed the Business Model Canvas for (Bawaba 2030). It summarizes the key partners, activities, resources, value propositions, customer segments, channels, customer relationships, cost structure, and revenue streams that define how the system operates and delivers value.

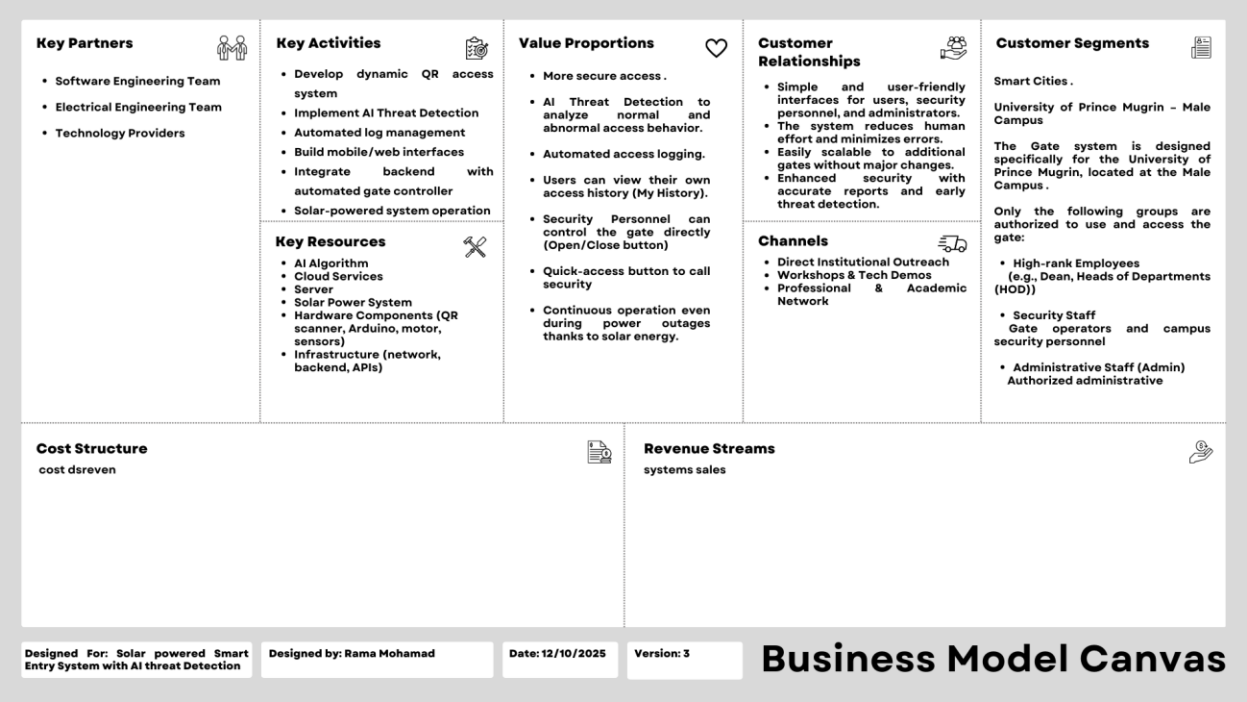


Figure 13. Business Model Canvas

CHAPTER 5: SYSTEM ANALYSIS

5.1 Overview

This chapter provides Software Engineering (SE) tasks involved in developing the system. These tasks include Requirement Analysis covering Need Analysis and Solution, System Stakeholders and Their Roles, Functional Requirements, System Requirements, Hardware Requirements, UI/UX Requirements, Non-Functional Requirements, and Realistic Constraints. In addition, this section presents the UML Diagrams (Use Case, Class, Activity, Sequence, and Deployment Diagrams) and the Overall Architecture, which includes evaluating alternative architectures, selecting the best alternative, finalizing the chosen design, and providing detailed design diagrams.

5.2 Glossary of Terms

Definitions and acronyms stated within the SE section:

Term / Acronym	Definition
QR Code	A two-dimensional barcode displayed on the user's mobile.
OTT	One-Time Token. A short-lived encrypted token that contains user ID, expiration time, and timestamp, and is embedded inside the QR code for secure, single-use access.
Redis Cache	An in-memory key-value data store used as a temporary database for QR code entries and validation logs before they are transferred to the main database.
Database	The permanent cloud-hosted database that stores users, roles, logs, anomaly labels, and other long-term system data.
AI	Artificial Intelligence a branch of computer science that enables systems to learn patterns from data and make automated decisions.
Threat Detection Model	The anomaly-detection component that analyzes validation logs and classifies each event as *normal* or *abnormal* based on learned access patterns.
Anomaly	Any access behavior that deviates from the user's normal pattern (e.g., unusual time, frequency, or location) and is flagged by the AI model.
Gate User	Any authorized person who uses the mobile/web app to generate QR codes and access the university gate (e.g., HOD, Dean, staff).
Security Personnel	Gate officers responsible for monitoring real-time logs, responding to alerts, and manually opening/closing the gate when required.
System Administrator (Admin)	The person who manages user accounts, access permissions, system logs, and analytics through the admin web panel.
MVP	Minimum Viable Product. The smallest functional version of the system that includes all must-have requirements needed for core gate access and logging.

FR	Functional Requirement. A requirement that specifies what the system shall do, such as QR generation, validation, logging, or sending commands to the gate.
NFR	Non-Functional Requirement. A requirement describing quality attributes of the system, such as availability, reliability, performance, security, or portability.
HR	Hardware Requirement. A requirement that specifies the needed hardware components, such as Arduino, QR scanner, DC motor, relay, sensor, solar panel, and battery.
Constraint	A realistic limitation that the system must operate under, such as using the existing Arduino controller or complying with university policies.
UX	User Experience. How easy, smooth, and comfortable it feels for the user to interact with the system.
UI	User Interface**.** The screens, buttons, and visual elements the user interacts with in the system.
REST API	A web service interface that uses HTTPS and JSON to allow the mobile app, backend server, AI model, and hardware controller to communicate.
JSON	JavaScript Object Notation. A lightweight data format used for structuring messages exchanged between system components over the REST API.
HTTPS	Hyper Text Transfer Protocol Secure. A secure version of HTTP that encrypts all communication between the client and server.
GMT+3	The time zone that is 3 hours ahead of Greenwich Mean Time; this is the standard time used in Saudi Arabia; all scheduled tasks (e.g., log transfer at 12:00 AM) are aligned to this standard time.
UML	Unified Modeling Language. A standardized modeling language used to create Use Case, Class, Activity, Sequence, and Deployment diagrams for the system.
KT-DA	Kepner-Tregoe Decision Analysis. A structured decision-making framework used to classify MUSTs and WANTs, weigh quality attributes, and select the best architecture alternative.
Agile	A methodology in the software development process in which software is created and developed through collaborative efforts within a designated, cross-functional team
Waterfall	This methodology is one of the most prominent traditional project management methodologies. It follows a linear and sequential process in

	implementing the project phases, as each phase in the project is completed prior to moving into the next phase, including collecting requirements, designing, development, testing, and deployment.
Kanban	It refers to a visual framework of the agile management and it promotes workflow improvements in minor and continuous changes of the executed process. Moreover, it is also known as "visual cards" or "sign" in Japanese.
Scalability	The system's ability to operate efficiently as it expands, allowing it to support multiple gates and multiple campuses without requiring major architectural changes or causing performance degradation.
Stakeholder	Parties and entities that affect and are affected by decisions, directions, procedures, objectives, policies and initiatives of the digital government and share some of their interests and outputs and are affected by any change that occurs in them.

Table 3. Glossary of Terms for SE

5.3 Requirement Analysis

In this part of the document, we applied the ISO/IEC/IEEE 29148:2018 standard, particularly the sections that define how requirements should be written, structured, and documented. First, we followed Clause 5.2.7 – Requirement Language Criteria, which emphasizes writing requirements using clear, testable, and unambiguous language. We also aligned our requirement phrasing with the commonly used "shall" requirement templates recommended in the standard. Also, we depended it from the SE 311 Slides regarding using boilerplate template for our functional requirements. Next, we applied Clause 5.2.8 – Requirements Attributes, which specifies that each requirement must be: Uniquely identified (e.g., FR-01, NFR-04, HR-03), Assigned a priority using an appropriate scale (Must-have, Nice-to-have, Optional), Categorized by type (Functional, Non-Functional, Hardware). In addition, we referenced Clause 9.2.3 – Definitions, where we constructed a glossary of terms to ensure consistent understanding of system concepts such as "QR code," "OTT," and other domain-specific terminology. For stakeholder analysis, we followed Clause 9.4.5 – Stakeholders, which requires listing all stakeholder classes and describing how each stakeholder interacts with and is related to the system. This is reflected in Table 3. System Stakeholders and Their Roles. Regarding system context and interfaces, we applied Clause 9.6.4 – Product Perspective, which defines how the system relates to external components and environments. Specifically: Clause 9.6.4.1 – System Interfaces We identified all system interfaces, such as the interface between the backend and the gate controller (Arduino), the interface between the mobile application and the server, and the interface between the AI model and the log-analysis module. Clause 9.6.4.2 – User Interfaces We described the logical characteristics of user interactions, such as QR generation on the mobile app, receiving access feedback, and displaying logs to Admin and Security Personnel. Clause 9.6.4.3 – Hardware Interfaces We defined the logical interaction between the software system and hardware components, including the QR scanner, DC motor, relay module, position sensor, and solar-powered control circuitry. By applying these sections of ISO/IEC/IEEE 29148:2018, we ensured that our requirements are well-structured, traceable, measurable, and aligned with international standards for system and software requirements engineering [5].

5.3.1 Need Analysis and Solution

In this part, we will be analyzing the needs of our project and suggesting potential solutions for the problems identified. These solutions will later be translated into system requirements across the different system entities.

Problem One: Lack of fully automated gate control and absence of smart log management

Many organizations' access systems still operate without fully automated gate control and without intelligent log management, meaning access events are not consistently captured or correlated with user identity. This creates blind spots in monitoring and weakens overall security.

So, we need a mechanism to automate gate control and ensure every access event is digitally captured and linked to the correct user.

Proposed Solution:

We can use a fully automated gate supported by digital logging and event tracking. It is worth noting that we can integrate QR-based access verification, which according to the comparison results in chapter 2, outperformed facial recognition in cost, speed, and feasibility for our environment. Based on reference [3], automated digital logs also enhance anomaly detection and provide accurate visibility into system activity.

Problem Two: Growth of new hacking tools and evolving attacker capability makes it easier to breach the gate workflow

Due to the rapid evolution of attackers' systems, unauthorized access may occur without alerting the security guard. We need a secure verification process that minimizes unauthorized access and a mechanism to detect suspicious behavior.

Proposed Solution

We can use encrypted, time-limited QR codes combined with a threat detection AI model to strengthen access validation. Let's say, for example, a doctor successfully enters through the gate at 10:15 AM, and then one minute later, the system receives another access request from the *same* doctor. Since a user cannot physically re-enter the gate immediately after just entering, this behavior is considered abnormal and the attempt would be flagged as suspicious.

Another example: a doctor who normally enters the campus at 7:00 AM and leaves around 3:00 PM, Monday through Thursday, suddenly attempts to access the gate on Friday at 8:00 PM. Such

an unusual access pattern strongly indicates abnormal behavior and should trigger an alert for further verification.

Based on reference [3], AI-driven log analysis significantly improves the detection of such anomalies and increases system resilience. However, we can't rely on manual observation or static codes, as they do not scale with modern threat capabilities and fail to detect subtle or sophisticated attack patterns.

Problem Three: Heavy reliance on the main power grid makes gates vulnerable during power outages

Traditional automatic gates become non-functional during power interruptions, increase operational costs, and fail to provide continuous secure operation.

We need a stable, resilient, and sustainable power source that ensures continuous functionality.

Proposed Solution

We can use a solar plate and battery system to maintain gate operations even during outages. It is worth noting that we also reduce long-term cost and dependency on the main grid. Based on reference [10], solar-powered automation solutions improve reliability and maintain uninterrupted access for critical systems.

Problem Four: Reduced system reliability, increased maintenance burden, and periods where the gate may stay open or unmonitored

These issues create security gaps and risk exposing the gate to unauthorized access.

Based on this problem we need a reliable, automated, and continuously monitored gate system.

Proposed Solution

We can use automated access validation, dynamic logging, and real-time alerts to ensure the gate is never left open or unmonitored. Let's say the system detects abnormal usage; an instant alert is dispatched to security personnel. Based on reference [3], real-time log management improves visibility, accountability, and event tracking in secure environments. But we can't depend solely on human presence, as manual monitoring is inconsistent and prone to error.

5.3.2 System Stakeholders and Their Roles

Here in this part, we define our stakeholders and present the following table, which shows the primary stakeholders involved in the System and summarizes each stakeholder's description and key responsibilities within the system.

Stakeholder	Description	Responsibilities
Gate Users	Authorized individuals who use the QR system to enter the university gate (e.g., HOD, DEAN, Admin Staff, Security Staff)	• Use the web-based application to generate and scan the QR code. • Follow system usage policies. • Report any access issues or failed entries. • Use the web-based application to see their previous entry log. • Use the web-based application to call security personnel when needed.
System Administrator (Admin)	The person responsible for managing and maintaining the system from the backend and web admin panel.	• Manage user accounts and access permissions. • Monitor system activity, logs, and QR usage statistics.
Security Personnel	Security officers stationed at the gate responsible for monitoring entry and responding to alerts.	• Monitor real-time access logs and gate status. • Verify user identity when needed. • Respond to AI-detected abnormal events or alerts. • Use the mobile to open or close the gate when needed.
Electrical Engineering Team	The team responsible for designing, building, and integrating the hardware components of the system.	• Design and implement the gate needs. • Set up and test the solar panel and battery system.

		• Ensure safe and reliable gate movement and feedback sensors. • Integrate hardware with the backend commands.
Software Engineering Team	The team responsible for the software components of the system, including backend, frontend, and integration.	• Develop the mobile/web application. • Implement QR generation, validation, and logging features. • Integrate the backend with the hardware controller. •Maintain system performance, security, and reliability.

Table 4. System Stakeholders and Their Roles

5.3.3 Functional Requirements

Here in this part, we are following a **Minimum Viable Product (MVP)** approach; therefore, we assigned a priority level to each requirement based on how essential it is for the system to function. The priority scale is as follows:
1 – Must-have: the requirement is essential and must be implemented.
2 – Nice-to-have: beneficial, but the system can still operate without it.
3 – Optional: acceptable if not implemented, since the system will still function under the MVP approach.

5.3.4 System Requirement

ID	Functional Requirement	Source	Priority
FR-01	When a user requests a QR code, the system shall generate a dynamic QR code containing an encrypted One-Time Token (OTT) that securely includes the user ID, expiration time, and timestamp.	Brainstorming, existing systems (google authenticator)	1
FR-02	When the system generates a QR code, it shall create a corresponding log entry that includes the QR code metadata and labels it with the log type “generation log.”	Brainstorming	3
FR-03	When a QR code is generated, the system shall store the QR code details in Redis cache.	Brainstorming	1
FR-04	The system shall automatically remove the QR code entry from the Redis cache after 1 minute to ensure security, prevent replay attacks, and minimize the window of unauthorized reuse.	Questionnaire, existing systems (google authenticator)	2
FR-05	After QR generation is done, the system shall display the QR code to the user on the mobile screen.	Questionnaire	1
FR-06	When the QR scanner reads a QR code at the gate, the system shall receive the QR code data.	Brainstorming	1
FR-07	When the QR code data is received, the system shall decode the QR code.	Brainstorming	1
FR-08	When the decoded QR code is available, the system shall check that the QR code format is valid and that the QR code is not expired.	Brainstorming	1

FR-09	If the QR code is valid and not expired, the system shall verify that the user linked to the QR code has access permission.	Brainstorming	1
FR-10	The system shall allow only one successful access per unique QR code.	Brainstorming	1
FR-11	When all access checks are passed, the system shall send “1” command to the gate controller (Arduino) and mark the access request as valid.	Hardware Integration need	1
FR-12	When any access check fails, the system shall send “0” command to the gate controller (Arduino) and mark the access request as invalid.	Hardware Integration need	1
FR-13	When the access request is approved, the system shall display a clear success message to the user.	Questionnaire	3
FR-14	When the access request is rejected, the system shall display a clear error message to the user.	Questionnaire	3
FR-15	When any access attempt occurs, the system shall create a log entry for that access attempt with a log type “validation log”.	Brainstorming	1
FR-16	When a log of the type “validation log” is being created, the system shall include user ID, gate ID, timestamp, log event type, and access status in the log entry	Brainstorming	1
FR-17	The system shall save the log entry in the Redis cache	Brainstorming	2
FR-18	The system shall automatically transfer all log entries from the Redis cache to the main database at 12:00 AM daily (GMT+3), and then clear the Redis cache after a successful transfer.	Brainstorming	2
FR-19	The system shall send log entries of the type “validation logs” to the AI model for analysis.	Brainstorming	1
FR-20	The AI model shall analyze each validation log and label it as either “normal” or “abnormal.”	Brainstorming	1
FR-22	When an event is labeled as abnormal, the system shall send an alert message to security personnel and store the anomaly label and alert details in the database.	Brainstorming	1
FR-23	The system shall provide Security Personnel with a main-page control button enabling gate opening and gate closing when required.	Brainstorming	1
FR-24	Admin users shall be provided with a dashboard that includes: (1) recent system activity, (2) an overview of all user accounts, and (3) system usage analytics	Brainstorming	1
FR-25	Security Personnel shall be able to receive and view real time server generated alerts regarding abnormal events	Brainstorming	1

FR-26	The system shall provide Admin users with a page that lists all system logs generated by all user roles, sorted in descending chronological order (newest to oldest)	Brainstorming	1
FR-27	A single Admin account shall be pre-defined during system development and retained during deployment. The system shall not permit creating Admin accounts through the standard user registration workflow.	Brainstorming	1
FR-28	Security Personnel accounts shall be created and assigned manually by the admin via a dedicated page for that purpose, to ensure controlled access to gate management features.	Brainstorming	1
FR-29	The system shall allow GateUser and Security Personnel to view their own activity history through a dedicated page labeled "My History," including access attempts and role-specific actions (e.g., gate opening and closing by Security Personnel).	Questionnaire	1
FR-30	The system shall automatically generate a log entry whenever Security Personnel open or close the gate.	Brainstorming	1
FR-31	The system shall provide a clearly accessible "Call Security" button that enables users to immediately call Security Personnel when needed.	Questionnaire	1
FR-32	Users must log in using their registered email address or mobile phone number. A One-Time Password (OTP) is sent for verification, and no system functionality is accessible until this authentication step is successfully completed.	Brainstorming	1
FR-33	The system shall provide users with a logout function that terminates the active session and returns the user to the login screen.	Brainstorming	1
FR-34	The system shall allow users to receive and view notifications, such as informational messages and success or failure feedback (e.g., "Gate opened successfully" and "Invalid QR code—try again")	Brainstorming	1

Table 5. System Requirements with Assigned Priority Levels Based on the MVP Approach.

5.3.5 Hardware Requirement

ID	Hardware Requirement	Source	Priority
HR-01	The system shall include a solar panel capable of supplying sufficient DC power to charge the battery and operate the control circuit under normal sunlight conditions.	Power system design	1
HR-02	The system shall include a rechargeable battery (24–30 V DC) to ensure operation.	Power system section	2
HR-03	The system shall include an Arduino Uno microcontroller as the main control unit to receive binary control commands (1 to open the gate, 0 to close the gate) from the backend system and drive the gate motor accordingly	Control unit design	1
HR-04	The system shall include a QR code scanner module compatible with Arduino Un and computer.	Input device section	1
HR-05	The system shall include a 12 V DC motor to open and close the gate within 2 seconds.	Actuator calculation section	2
HR-06	The system shall include a relay module (2 channels) to control the DC motor direction (open/close) safely.	Relay module section	2
HR-07	The system shall include a sensor to provide gate position feedback to the controller.	Feedback diagram	1
HR-08	The system shall include a motor driver interface to isolate Arduino Uno pins and handle motor current load.	Circuit simulation description	3
HR-09	The system shall include connectors, wiring, and protective casing to ensure stable connections and environmental protection.	Implementation design	1
HR-10	All electrical components and wiring shall comply with NFPA 70 (National Electrical Code) and relevant local safety standards for low-voltage DC systems.	Safety & compliance requirements	1

Table 6. Hardware Requirements with Assigned Priority Levels Based on the MVP Approach.

5.3.6 UI/UX Requirements

ID	Functional Requirement	Source	Priority
FR1	Allow users to create an account using university email/phone	User Registration	1
FR2	Require OTP verification via email/SMS	User Registration	1
FR3	Prevent registration with existing email/phone	User Registration	1
FR4	Login using university email/phone and password	Login	1
FR5	Send OTP after login credentials	Login	1
FR6	Login only after correct OTP	Login	1
FR7	Generate dynamic QR every 15–30 seconds	QR Management	2
FR8	QR contains secure encoded user data	QR Management	2
FR9	Display countdown timer for QR validity	QR Management	2
FR10	Manual QR refresh button	QR Management	3
FR11	Prevent QR reuse (anti-replay)	QR Management	1
FR12	Gate system scans QR and sends result	Gate Access	1
FR13	Display access status (Granted/Denied/Error)	Gate Access	1
FR14	Display gate opening animation/message	Gate Access	3
FR15	Dashboard shows account status, QR validity, last entry, notifications	Dashboard	2
FR16	Dashboard alerts user if QR expired/invalid	Dashboard	2
FR17	View personal info (name, email, phone, role)	Profile	3
FR18	Change password	Profile	2
FR19	Update profile photo	Profile	3

FR20	Send notifications (entry success/fail, suspension, updates)	Notifications	2
FR21	View notifications in app/web	Notifications	3
FR22	Logout page options (logout / return dashboard)	Logout	3
FR23	Confirmation popup before logout	Logout	3
FR24	Admin access to web admin panel	IT Management	2
FR25	Admin manages user accounts	IT Management	1
FR26	Admin views QR usage statistics	IT Management	2
FR27	Admin manages gate settings	IT Management	2
FR28	Officer views gate logs real-time	Security Gate Officer	2
FR29	Officer checks user identity if needed	Security Gate Officer	3
FR30	Officer monitors gate and suspicious activity	Security Gate Officer	2
FR31	Team members manage system configuration	Developers/Admins	3
FR32	Team members view logs for debugging	Developers/Admins	3
FR33	Team members access test/dev dashboards	Developers/Admins	3
FR34	Gate receives QR scan data	Gate System (Hardware)	1
FR35	Gate validates QR with backend	Gate System (Hardware)	1
FR36	Gate opens/closes automatically	Gate System (Hardware)	1
FR37	Gate reports status to app/web	Gate System (Hardware)	2

Table 7. UI/UX Requirements with Assigned Priority Levels Based on the MVP Approach.

5.3.7 Non-Functional Requirements

NFR#	Non-Functional Requirements Description	Source	Priority
NFR-01	The system shall maintain 99.99% uptime to ensure continuous operation of the System during university hours.	Availability	1
NFR-02	The system shall recover from unexpected failures within 15 seconds, ensuring that no access logs are lost.	Reliability	1
NFR-03	The system shall be capable of operating on multiple gates and across both university campuses if needed, without architectural changes and with consistent functionality.	Scalability	1
NFR-04	The system shall process each QR scan and return an access decision within ≤ 1 second.	Performance	1
NFR-05	The system shall encrypt the One-Time Token (OTT) before appending it to the static URL, using an approved encryption method (e.g., AES-256), to ensure an additional layer of security.	Security	2
NFR-06	The system shall provide clear access feedback (Success/Denied) to users within 1 second.	Usability	1
NFR-07	The AI anomaly detection model shall achieve at least 95% accuracy in identifying abnormal access behavior.	Accuracy	1
NFR-08	The system shall ensure seamless interoperability between the on-premise gate hardware, the mobile application, the cloud-hosted AI model, and the cloud main database through standardized, versioned REST APIs using JSON over HTTPS.	Interoperability	2
NFR-9	System modules shall be designed with low coupling so that modifications affect **no more than 10%** of other components.	Maintainability	1
NFR-10	The web application shall run consistently across all modern mobile web browsers (e.g., Chrome, Safari,	Portability	2

	Firefox, Edge) on both Android and iOS devices without requiring code changes.		

Table 8. Non-Functional Requirements with Assigned Priority Levels Based on the MVP Approach.

5.3.8 Realistic Constraints

ID	Constraint Description
CON-01	The system shall operate with the existing gate controller hardware (Arduino-based), and the command protocol cannot be changed.
CON-02	The system shall function within the university's network policies, including firewall rules, port restrictions, and security policies.
CON-03	The web application shall run only on modern mobile browsers and will not support desktop environments.
CON-04	All scheduled tasks, including log transfers and token expiry, shall follow the university's standard timezone (GMT+3).
CON-05	The AI model and main database shall be deployed on the approved designated cloud provider (e.g. AWS).
CON-06	The system must comply with university cyber-security policies, including data encryption, user privacy, and secure communication requirements.
CON-07	The system shall be deployable on multiple gates using the same backend and architecture without requiring redesign.

Table 9. System Constraints

5.4 UML Diagram

In this part of the document, we applied the official UML relationship types and notation defined in the OMG UML 2.5.1 standard [6], and we modeled each diagram using its correct structural and behavioral constructs. For example: in the Sequence Diagram, as seen in Figure (18), we used standardized UML elements such as lifelines, activation bars, synchronous messages, return messages, and the alt combined fragment to represent conditional logic. Also in the Activity Diagram, shown in Figure (25), we applied UML behavioral relationships, including control flows, decision nodes, merge nodes, and clearly defined initial and final nodes. In the Deployment Diagram, as seen in Figure (33), we employed node relationships, execution environments, device nodes and communication paths that align with UML's deployment modeling rules. In the Class Diagram, presented in Figure (16), we followed the official UML structural relationships such as association, generalization (inheritance), and interface usage, ensuring that each class interaction follows UML 2.5.1 semantics.

5.4.1 Overall Use Case

The system's use cases are separated into multiple diagrams to improve clarity and readability. This first diagram focuses on the *common system functions* used by several roles (Gate User, Security Personnel, and Admin) (see figure 7). More specialized and role-specific use cases are presented in the subsequent diagrams. As shown in Figure (8).

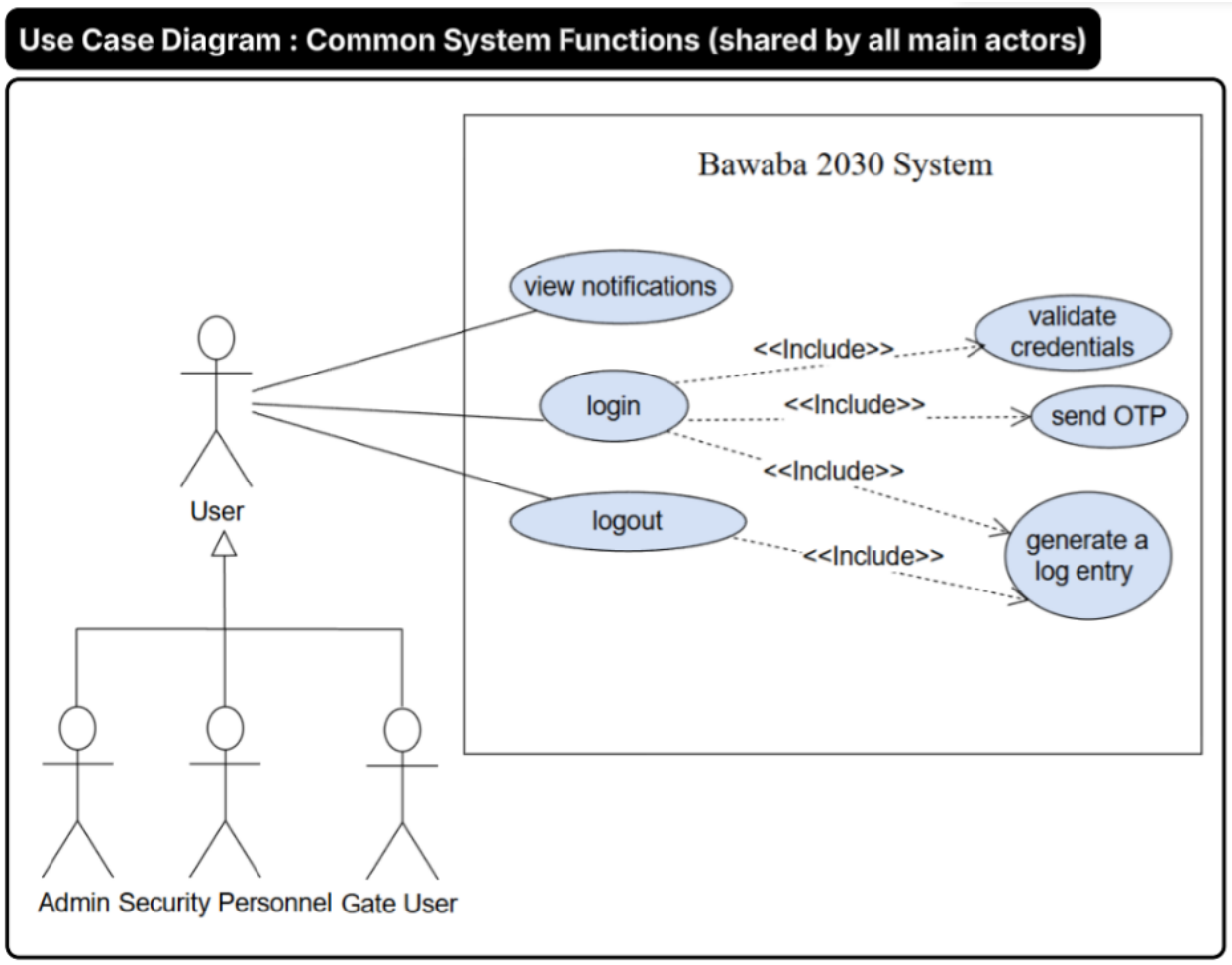


Figure 14. Use case diagram showing common system functions shared by all main actors.

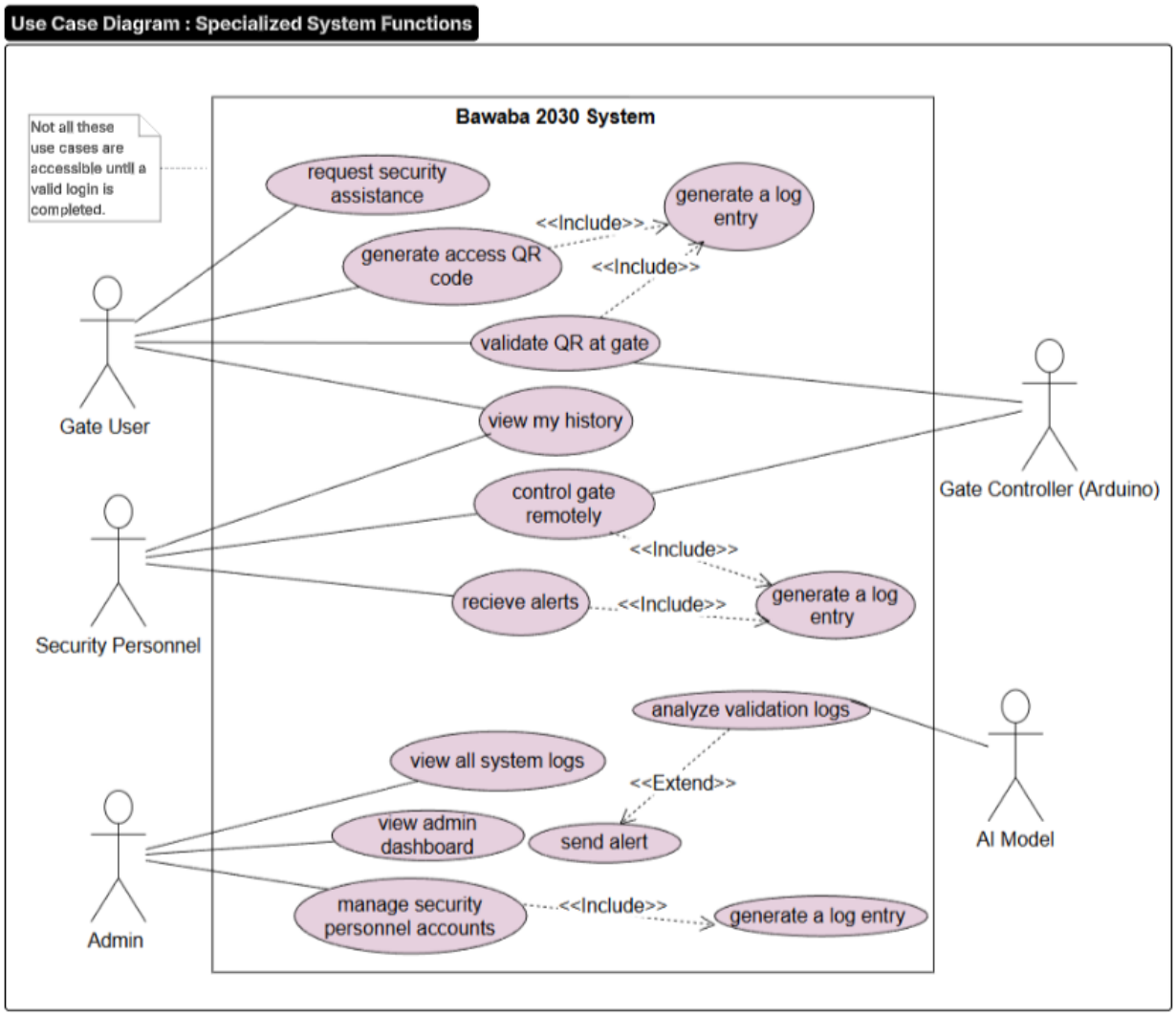


Figure 15. Use case diagram depicting specialized functionalities

5.4.1.1 Use Case Discerptions:

Use case name	Actors	Requirements Covered	Why it was picked	Description
Login	Gate User, Security Personnel, Admin	FR-32	Authentication is a must-have MVP requirement that protects all system functions and prevents unauthorized access.	User enters email/phone + password, receives OTP, then access is granted after OTP verification.
Logout	Gate User, Security Personnel, Admin	FR-33	It ends the session to reduce unauthorized access risk on shared devices.	User logs out, the active session is terminated, and the system returns to the login screen.
Generate Access QR Code	Gate User	FR-01, FR-02, FR-03, FR-04, FR-05	It is the main access method; the QR must be secure, short-lived, and auditable.	System creates an encrypted one-time QR with a token, caches it in Redis, logs it, and shows to the user.
Validate QR at Gate	Gate User, Gate Controller (Arduino)	FR-06, FR-07, FR-08, FR-09, FR-10, FR-11, FR-12, FR-13, FR-14	It is the core decision point that approves/denies entry and triggers the gate action.	System decodes QR, checks expiry/format/permission/one-use, then sends 1 (open) or 0 (deny) to Arduino.
View History	Gate User, Security Personnel	FR-29	It provides transparency by allowing users to review their own activities and role-specific actions recorded by the system.	User views their personal activity history, which includes system-recorded actions based on their role (e.g., access attempts or gate control events).
View Notifications	Gate User, Security Personnel, Admin	FR-34	It supports awareness by showing important system/security updates to users	User opens notifications page to view latest alerts/messages sent by the system.
Request Security Assistance	Gate User	FR-31	Provides a low-cost safety mechanism allowing users to reach security	User taps “Call Security” to immediately contact Security Personnel for assistance.

			without permanent on-site staffing.	
Receive Alerts	Security Personnel	FR-22, FR-25	Real-time alerts are essential for fast response to abnormal events, supporting secure gate monitoring in the MVP.	System pushes real-time alerts to Security Personnel for abnormal events detected by the AI.
Control Gate Remotely	Security Personnel	FR-23, FR-30	Manual gate control is required in exceptional situations to ensure safety and continuous operation under the MVP scope.	Security Personnel remotely open or close the gate through the system, and each action is automatically logged.
View All System Logs	Admin	FR-26, FR-18	Admins must audit and monitor system activity; logs enable accountability and troubleshooting under MVP scope.	Admin views all logs sorted newest-to-oldest, including daily transferred records from Redis to DB.
View Admin Dashboard	Admin	FR-24	It provides a single place to monitor activity, accounts, and system usage analytics.	Admin opens dashboard to see recent activity, account overview, and usage analytics.
Manage Security Personnel Accounts	Admin	FR-28	Security accounts must be controlled by admin to protect gate management capabilities and reduce misuse risk.	Admin creates/assigns Security accounts manually; via a dedicated page
Analyze Validation Logs	AI Model	FR-19, FR-20	AI-based labeling is required to detect abnormal access patterns automatically	AI receives validation logs and labels each event as normal or abnormal.
Send Alert	AI Model, Security Personnel	FR-22, FR-25	It turns “abnormal” detection into an actionable response for campus security.	If abnormal is detected, system alerts Security Personnel and stores anomaly + alert details in DB.

Table 10. Use Case Discerptions

5.4.2 Overall Class Diagram

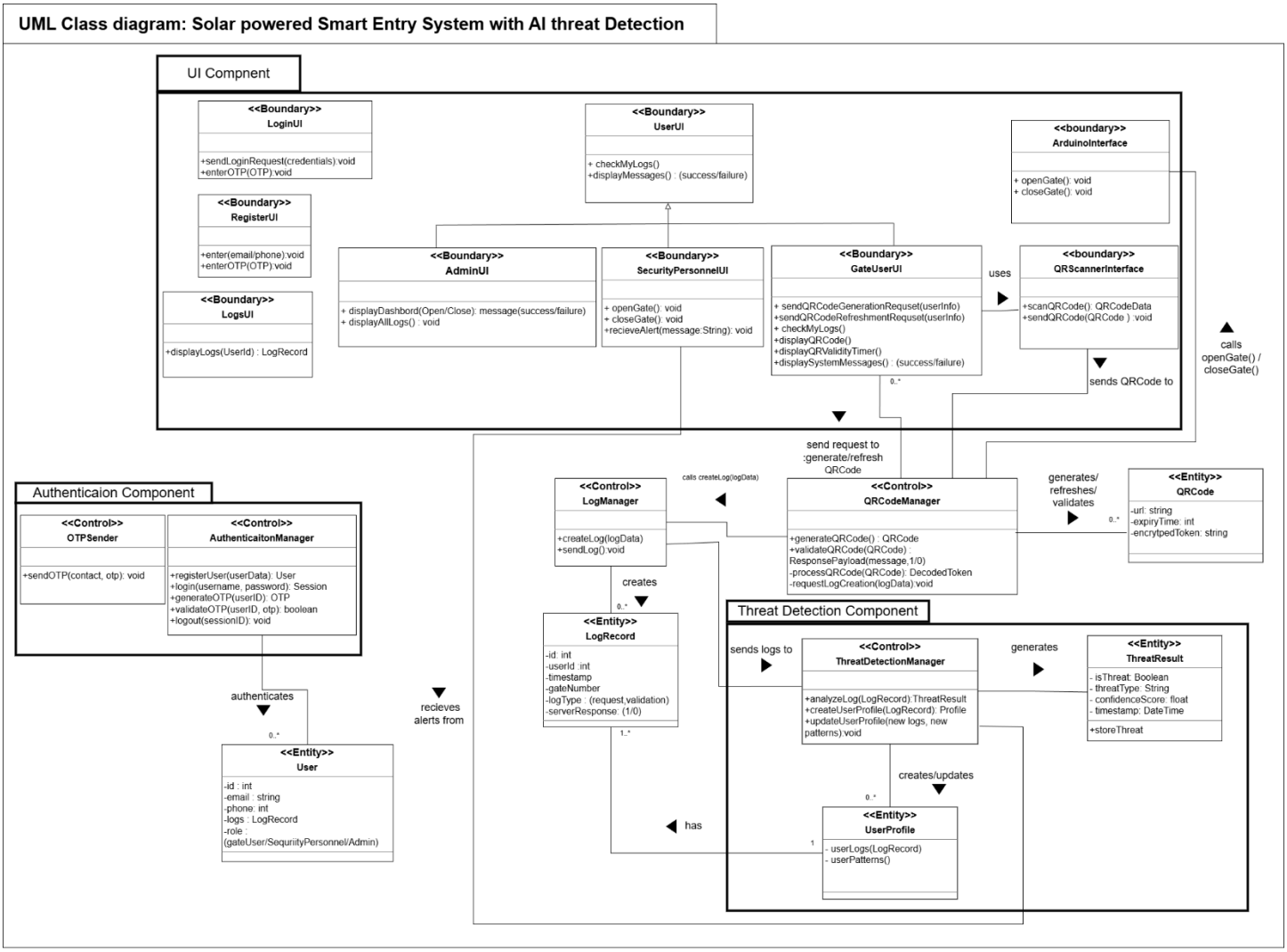


Figure 16. Overall, Class Diagram

5.4.3 State machine Diagram

state machine diagram : QR Code Validation Lifecycle (QRCodeManager class)

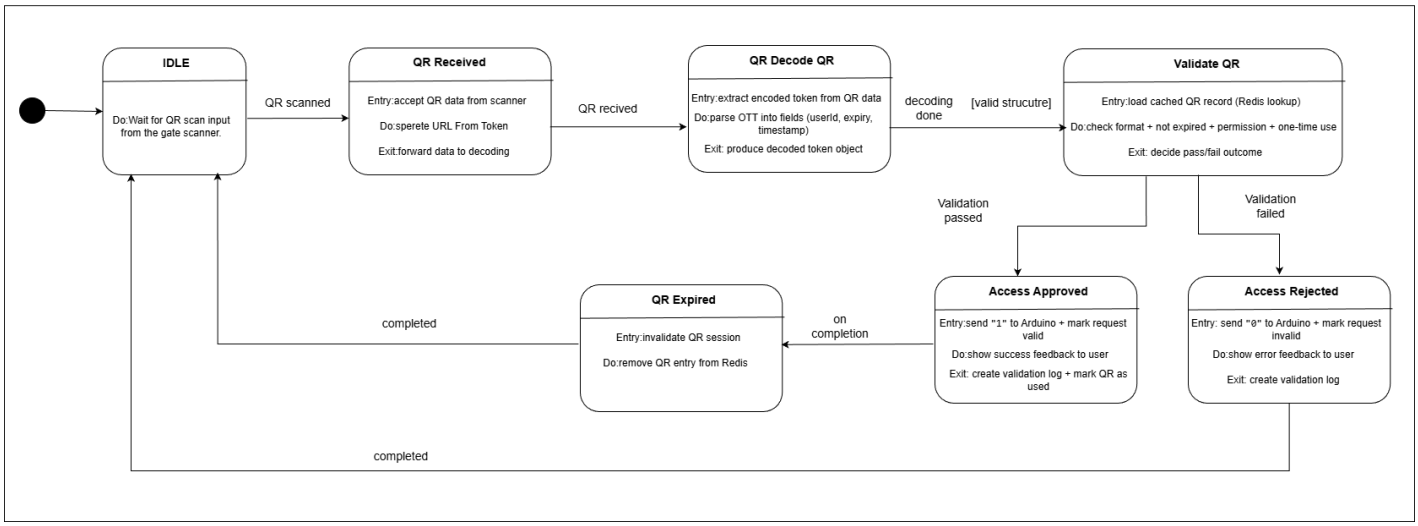


Figure 17. State machine Diagram

5.4.4 Sequence Diagrams

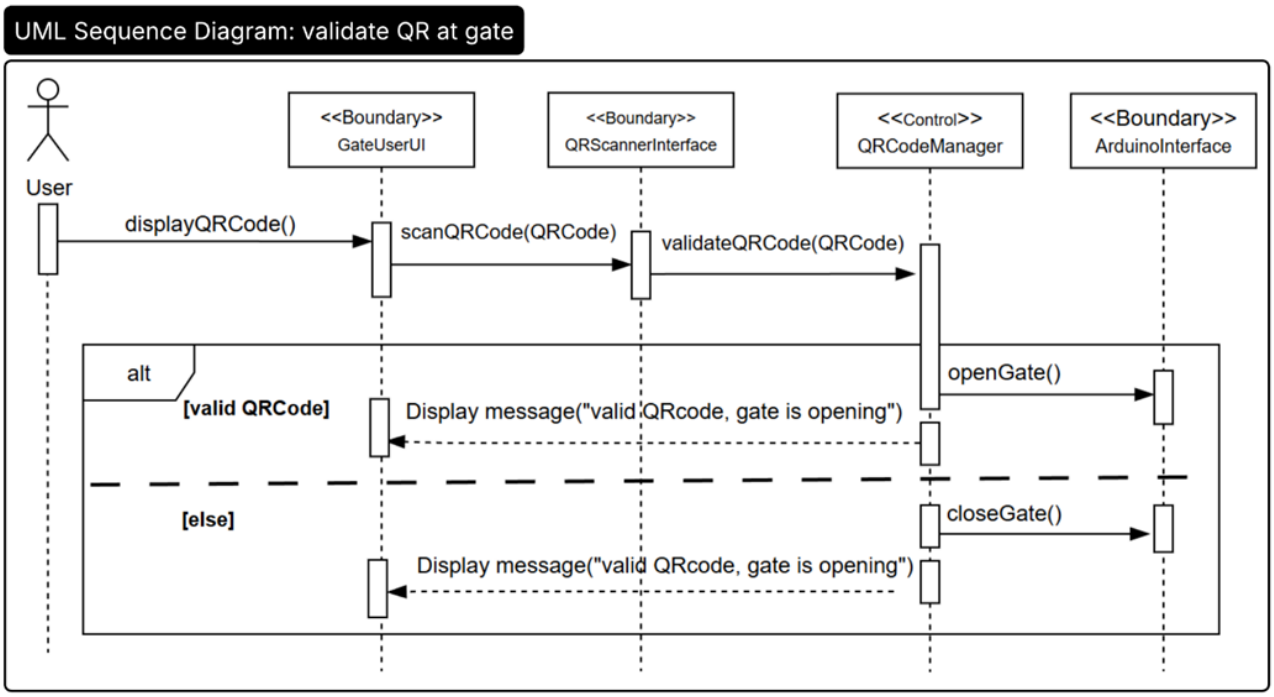


Figure 18. UML Sequence Diagram for QR Handling

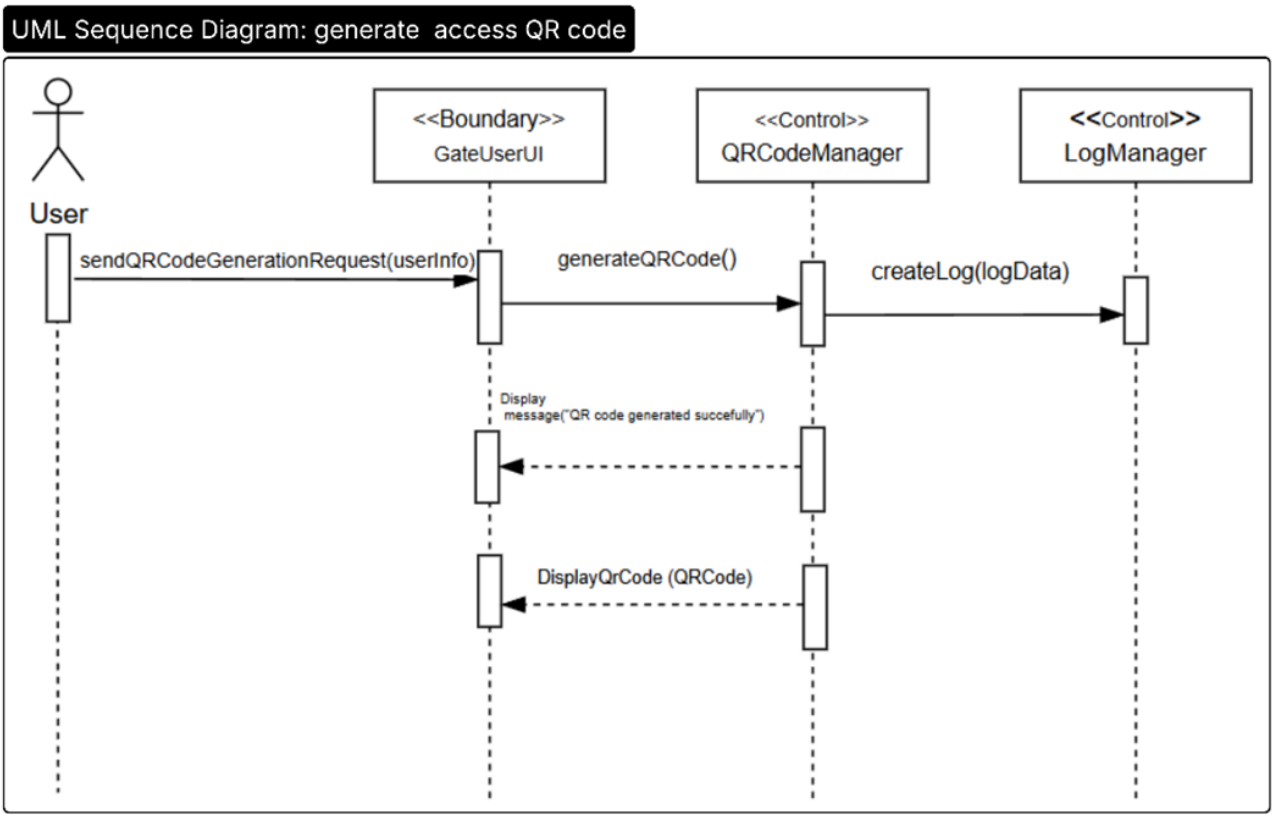


Figure 19. UML Sequence Diagram for QR Code Generation

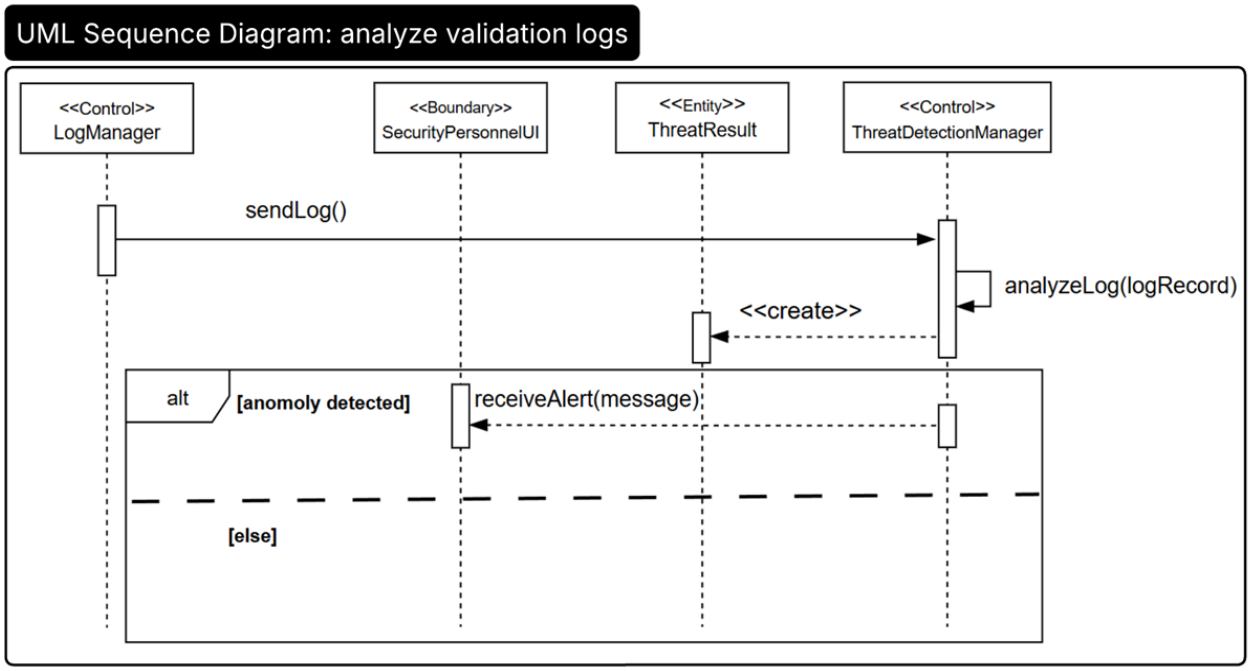


Figure 20. UML Sequence Diagram for Checking Anomaly Logs

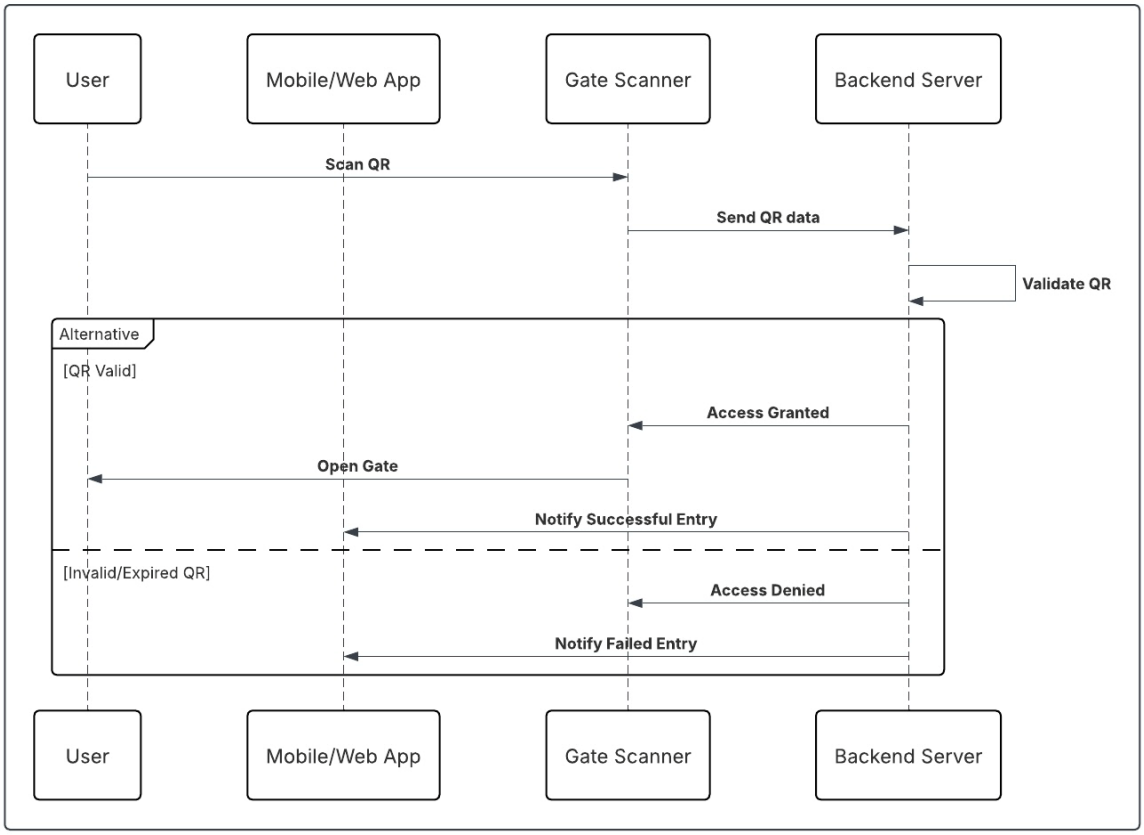


Figure 21. User Registration Sequence Diagram

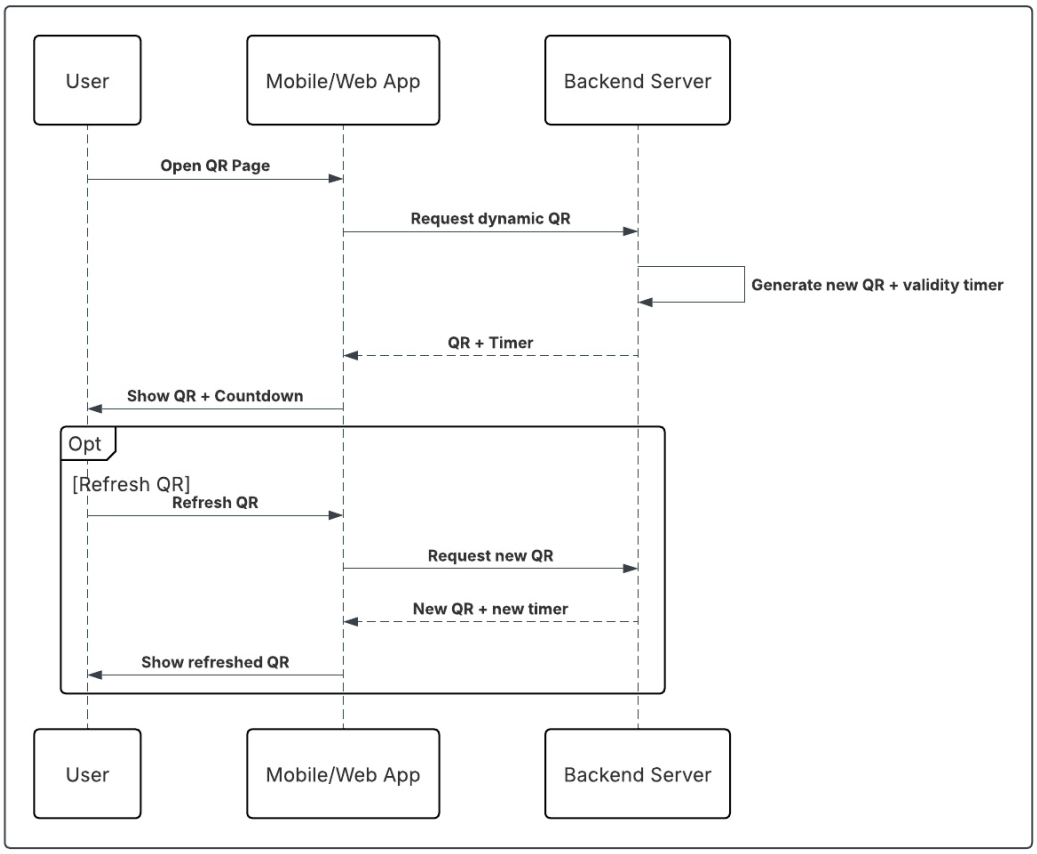


Figure 22. User Login Sequence Diagram

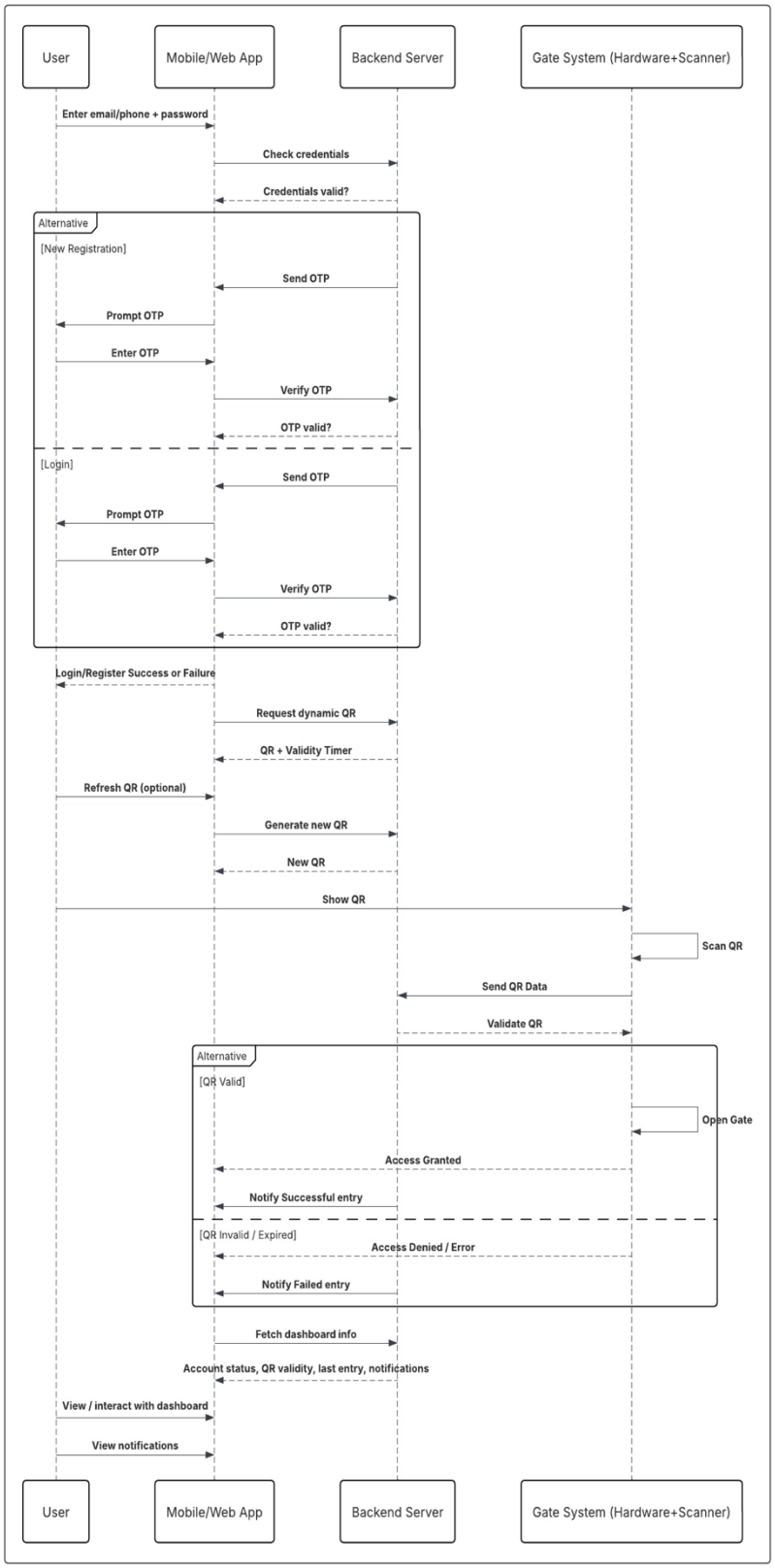


Figure 23. Secure User Authentication via OTP Sequence Diagram

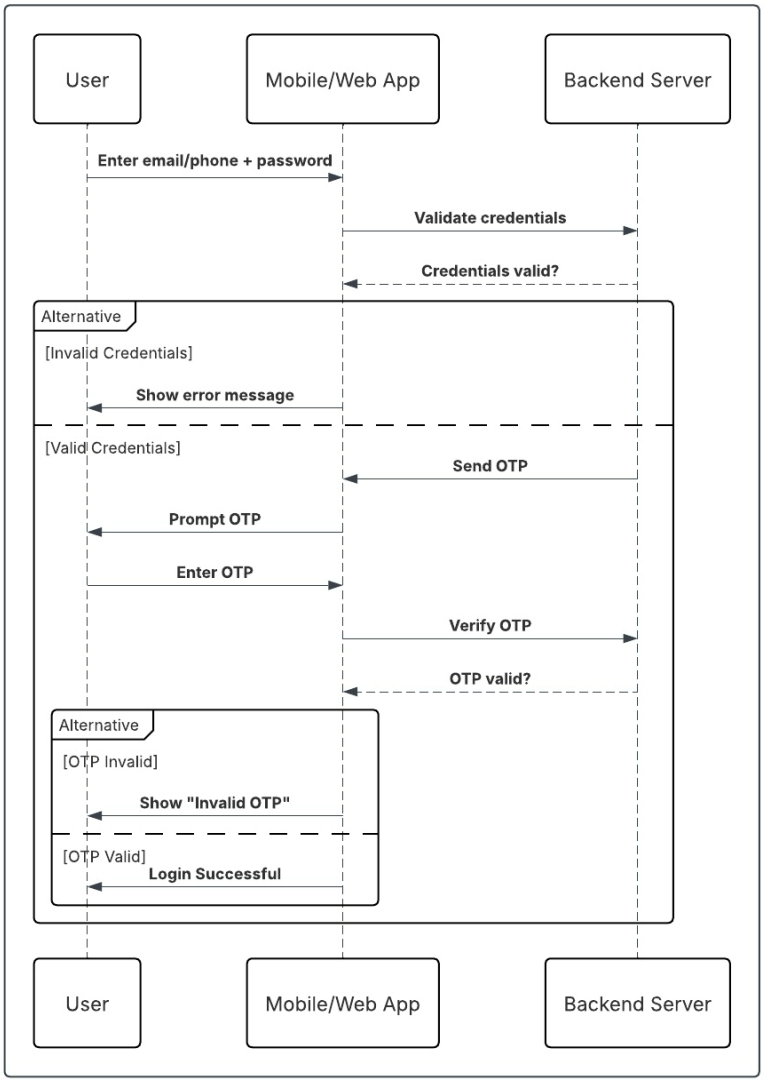


Figure 24. End-to-End Secure Access Flow Sequence

5.4.5 Activity Diagrams

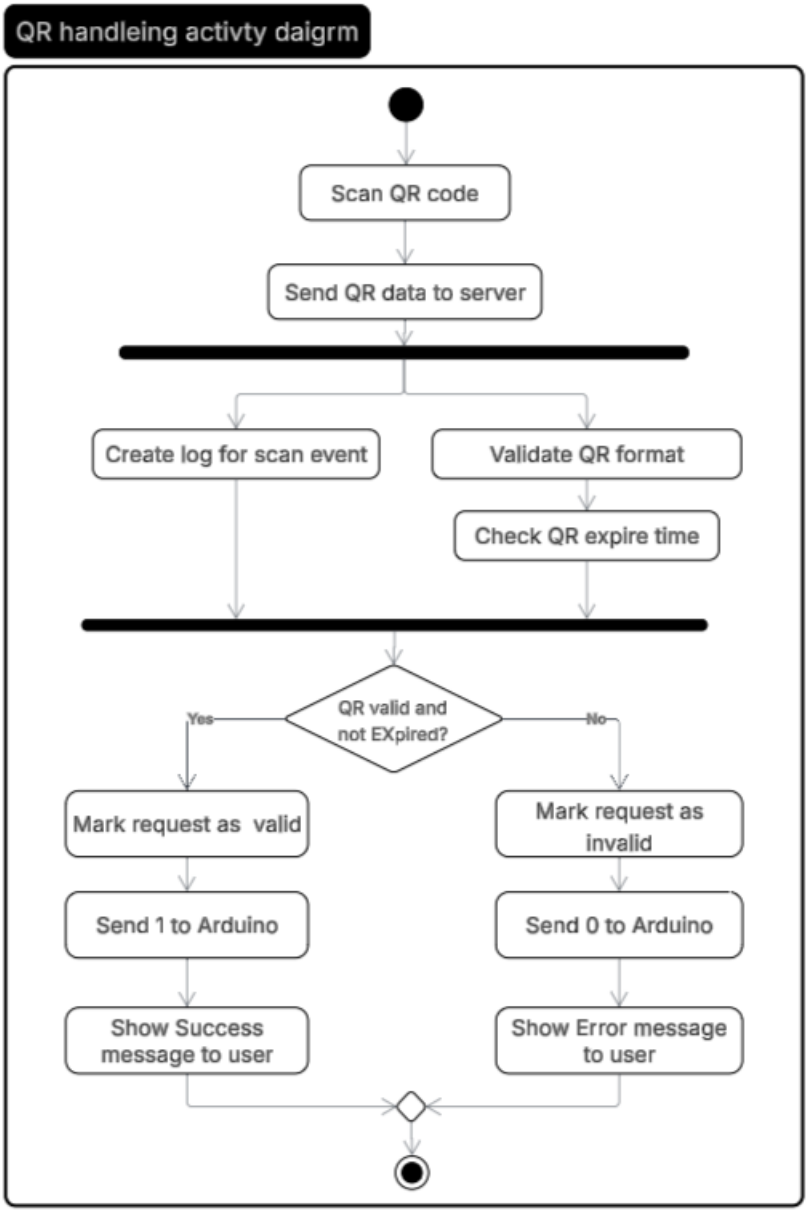


Figure 25. QR Handling Activity Diagram

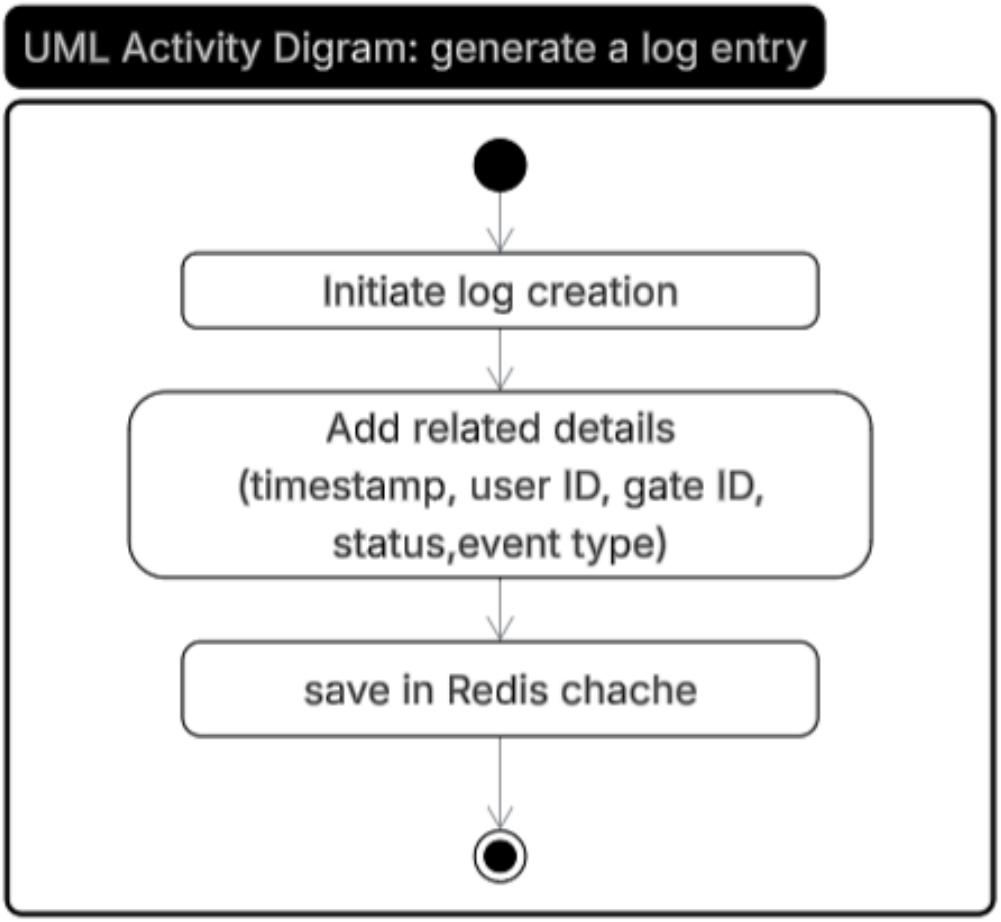


Figure 26. Create Log for Event Activity Diagram

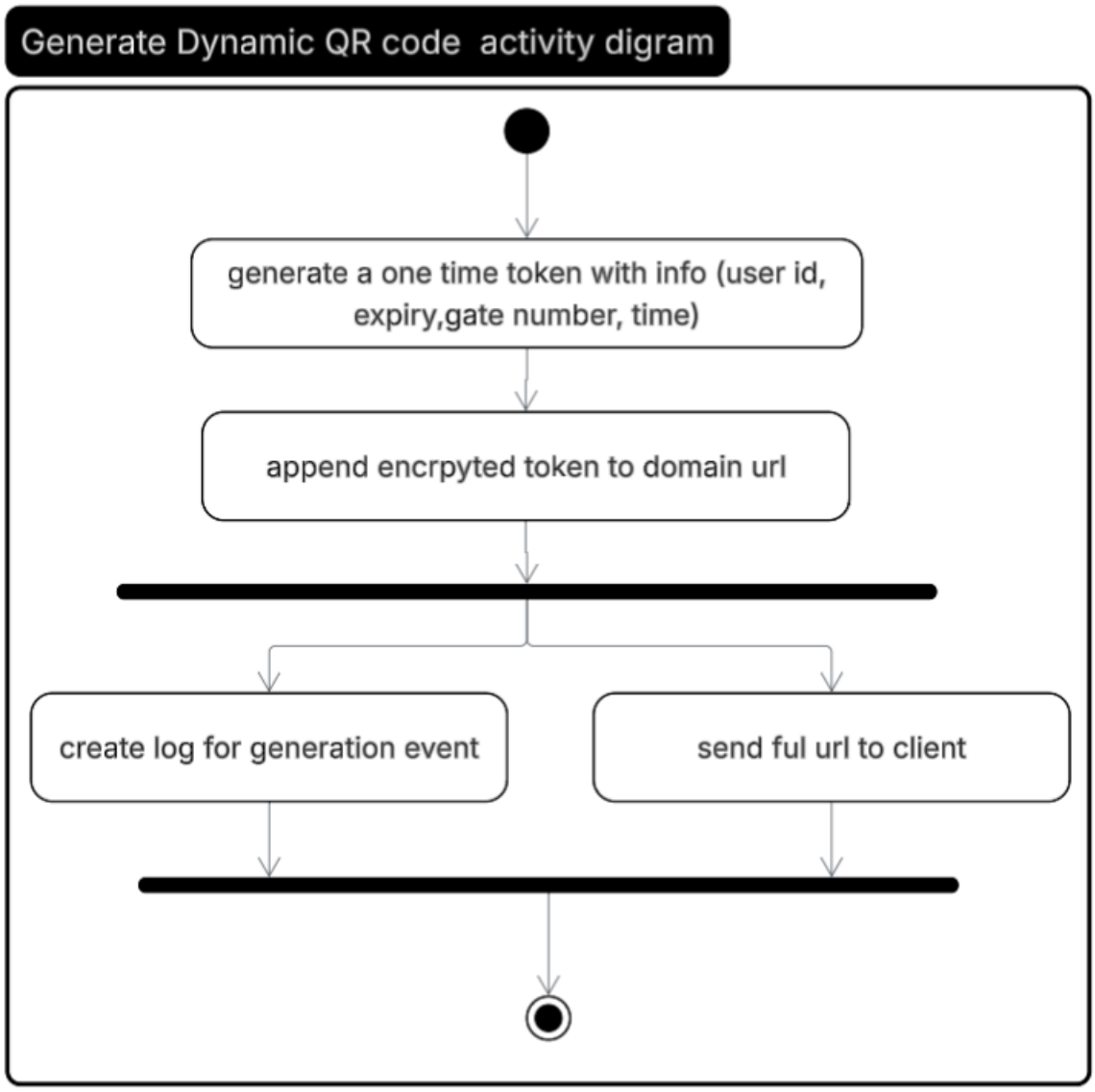


Figure 27. Generate Dynamic QR Code Activity Diagram

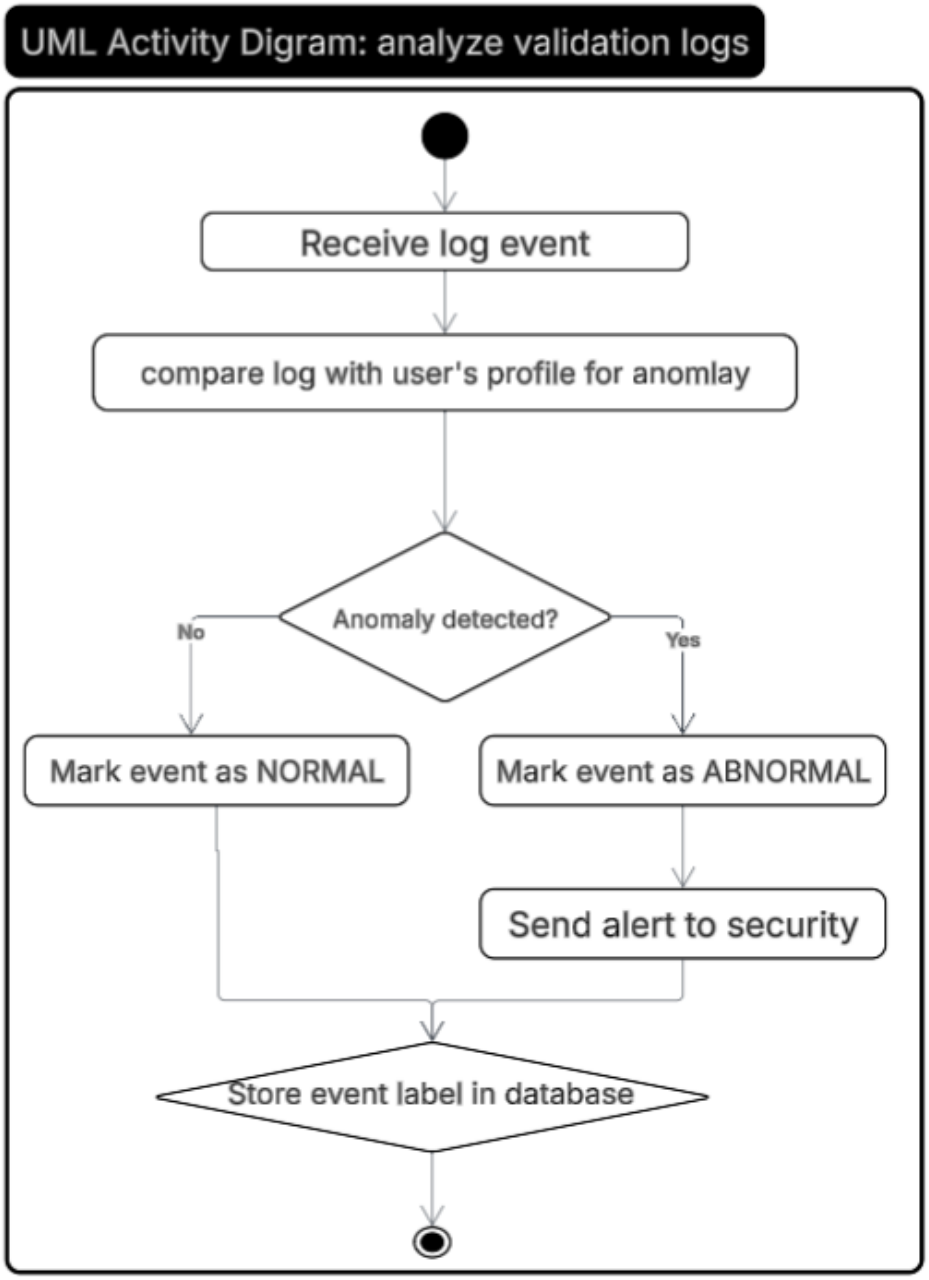


Figure 28. AI Checking for Anomaly Activity Diagram

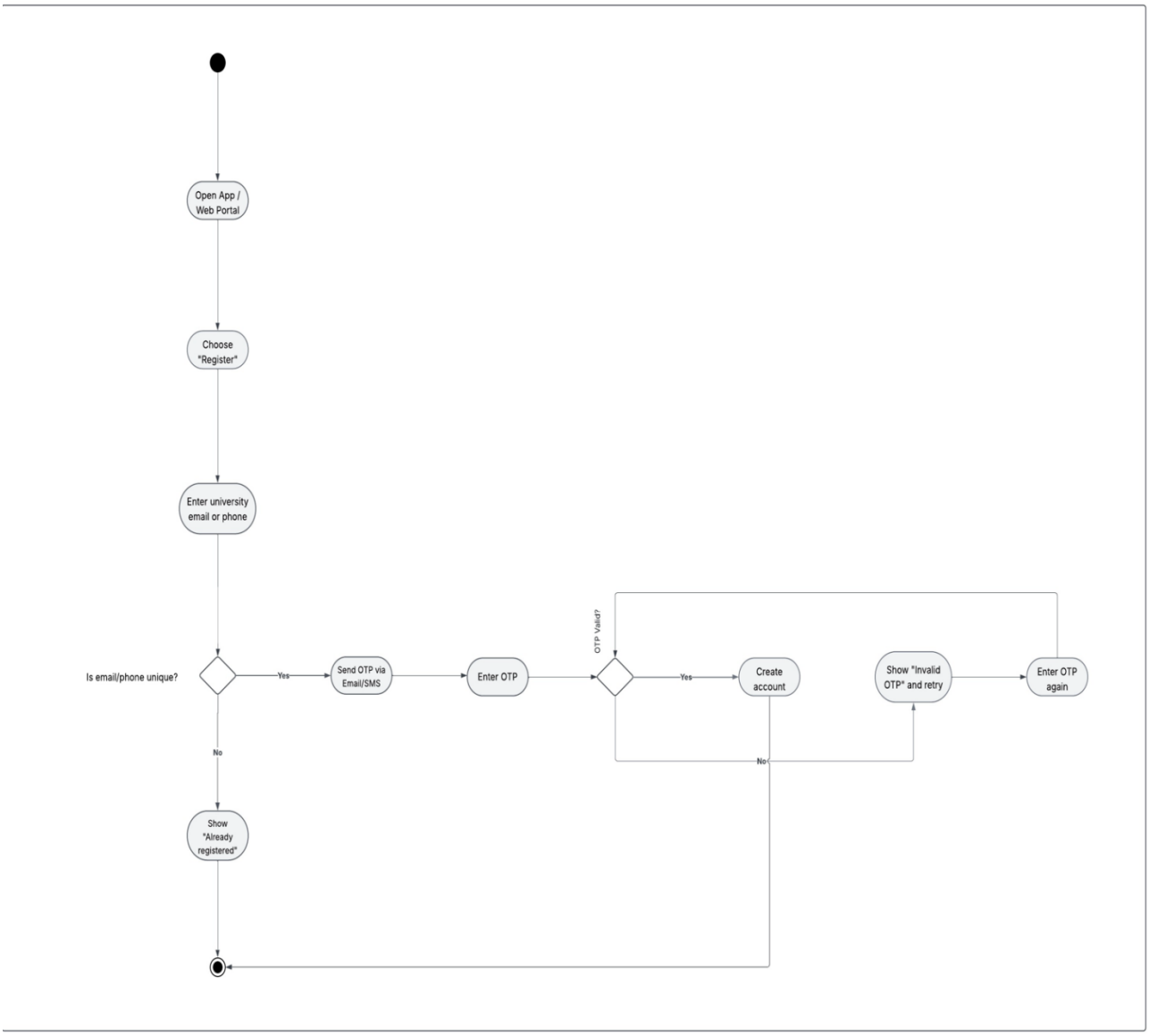


Figure 29. User Registration Activity Diagram

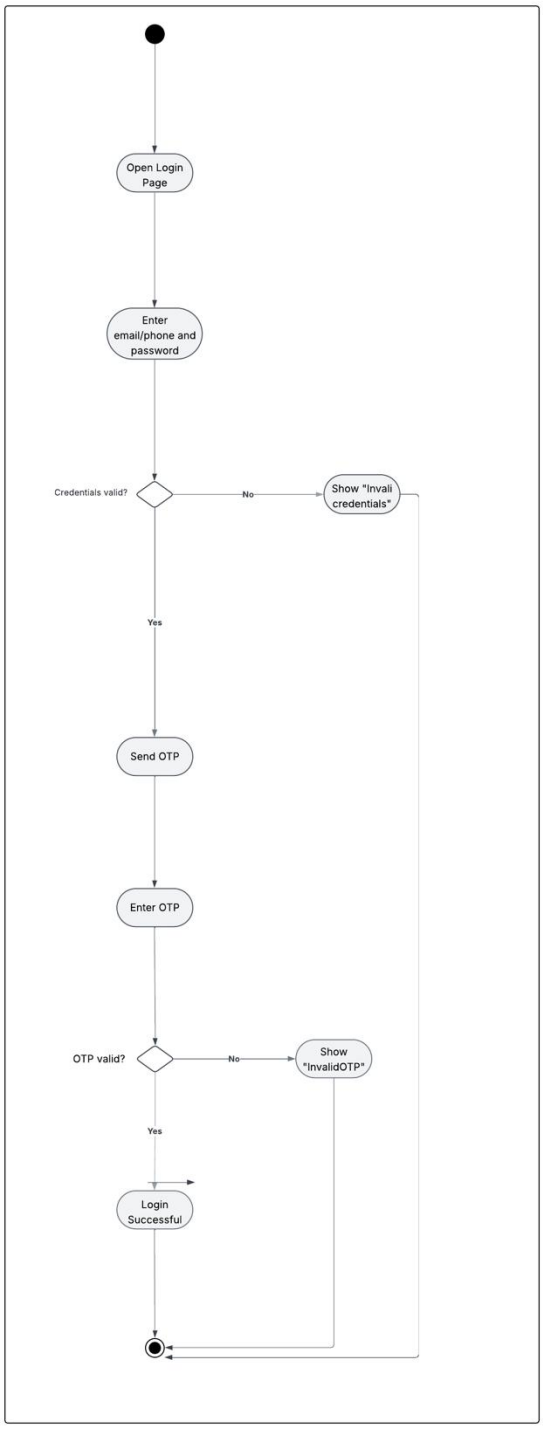


Figure 30. User Login Activity Diagram

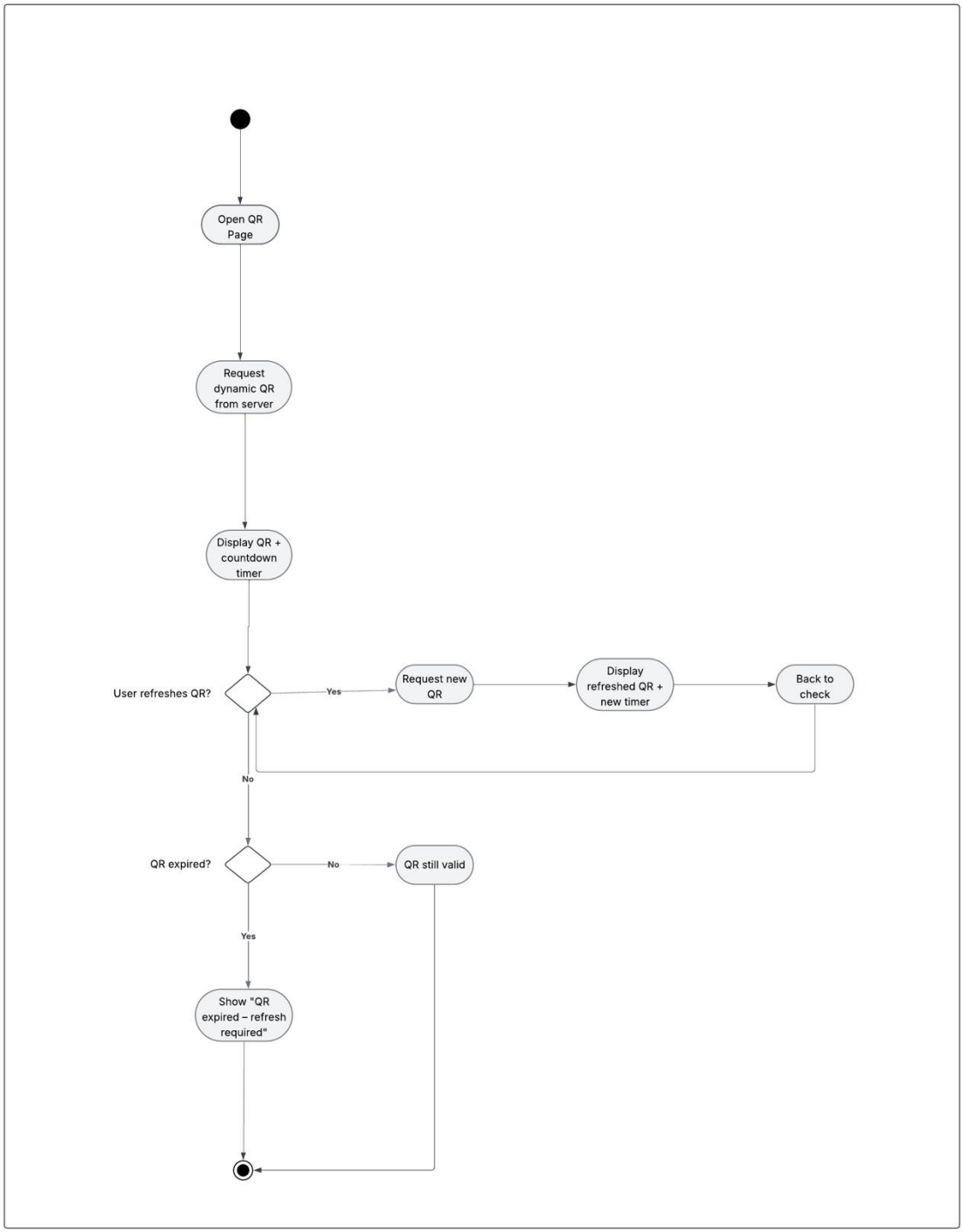


Figure 31. Dynamic QR code Mobile App Flow Activity Diagram

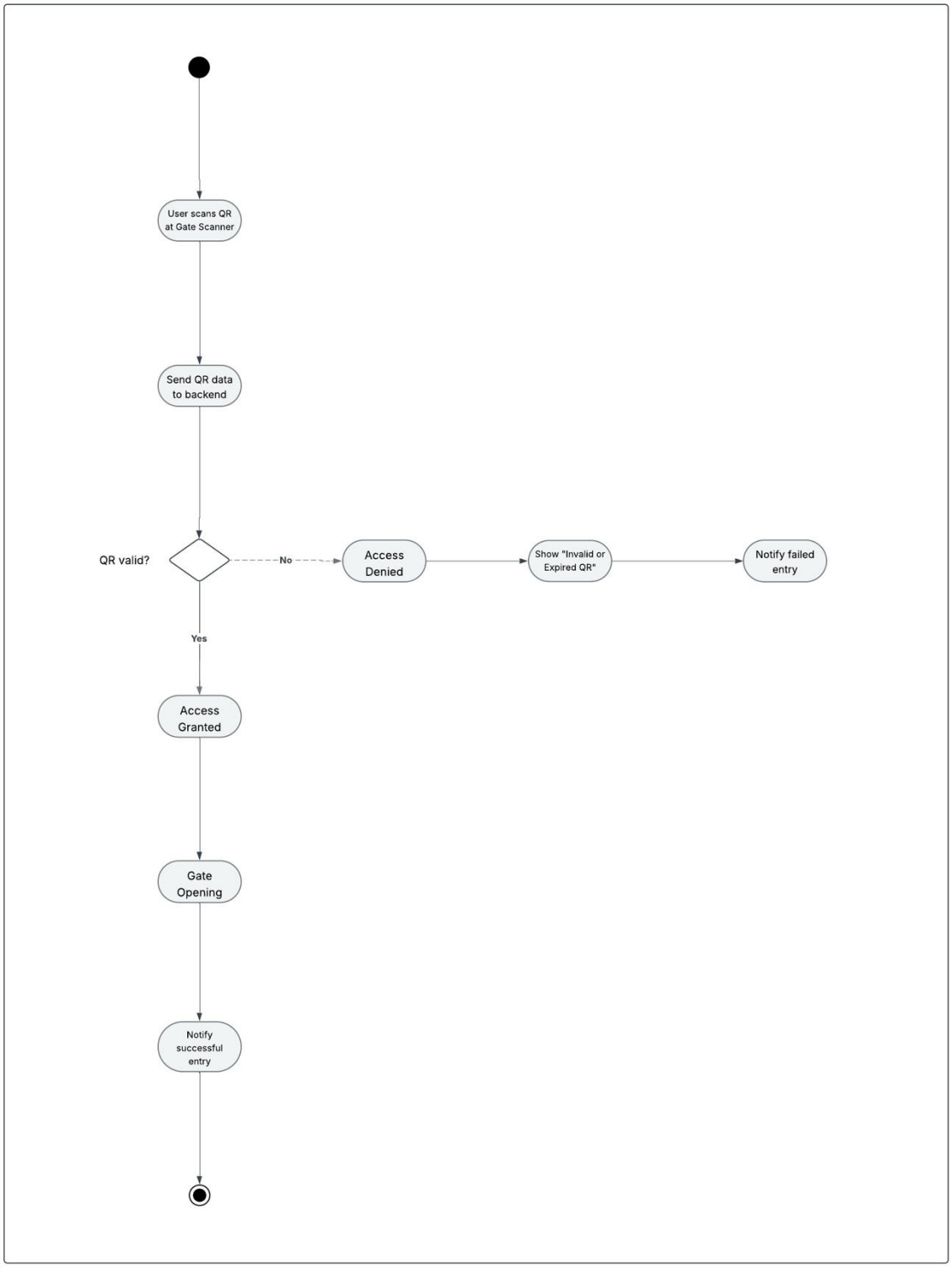


Figure 32. Gate Access Activity Diagram

5.4.6 Overall Deployment Diagram

This is another view of our project that reflect a physical hardware device and environment where software components are deployed.

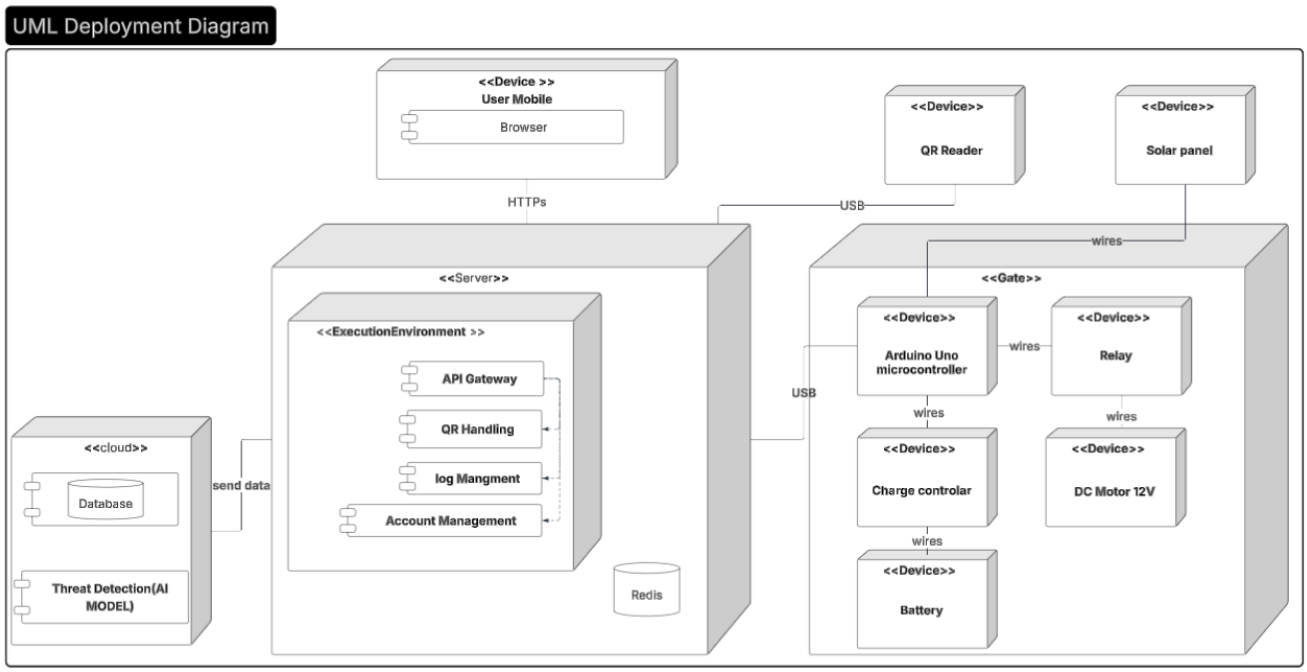


Figure 33. Deployment Diagram showing the physical distribution of system components.

5.5 Overall Architecture

In this part of the document, we adopted the Kepner-Tregoe Decision Analysis framework as the guiding method for evaluating and comparing our architectural alternatives. This structured approach allowed us to classify objectives into MUSTs and WANTs, assign weights to quality attributes, screen infeasible architectures, and ultimately identify the architectural design that best satisfies our system requirements.

5.5.1 Alternative Architectures

Here we have alternative architectures in which we will later, as a team, make a meeting to evaluate the 3 architectures; hence, once we decide on a particular architecture based on specific quality attribute to evaluate architecture and select the best architecture alternative.

5.5.1.1 First Alternative

The first alternative design for this System adopts the Event-Driven Microservices architecture.

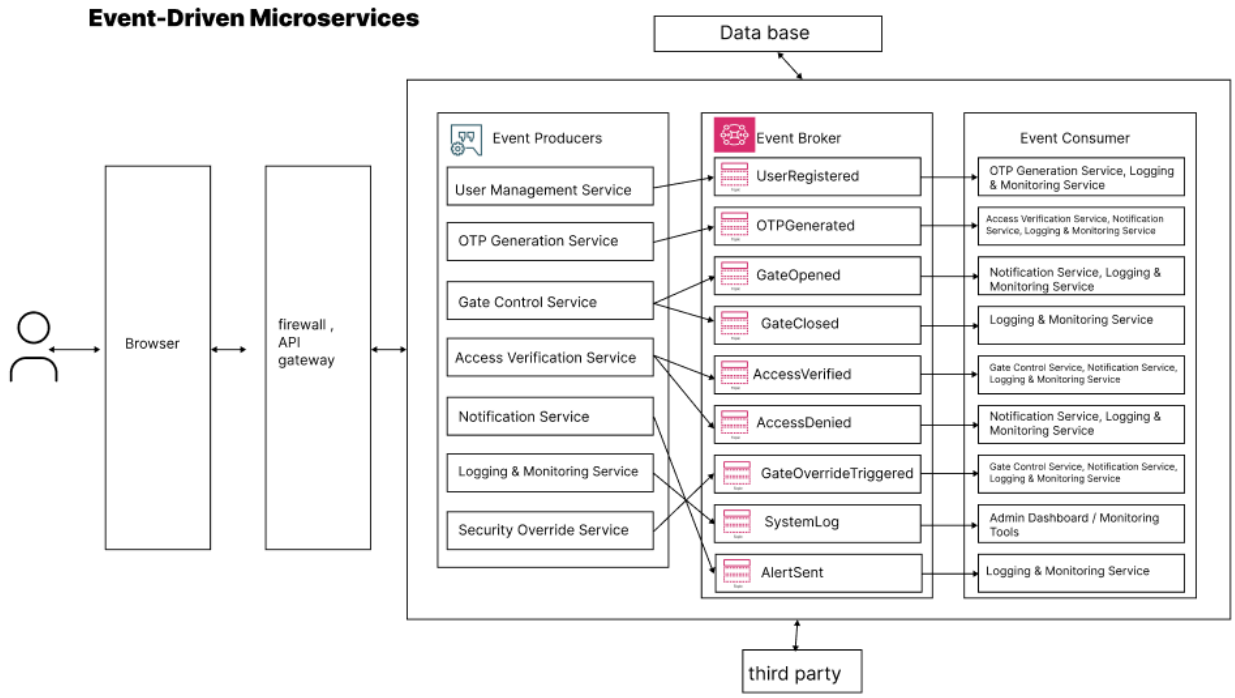


Figure 34. Alternative architecture [1]

5.5.1.2 Second Alternative

The second alternative design for this System adopts the 3-Tier architecture.

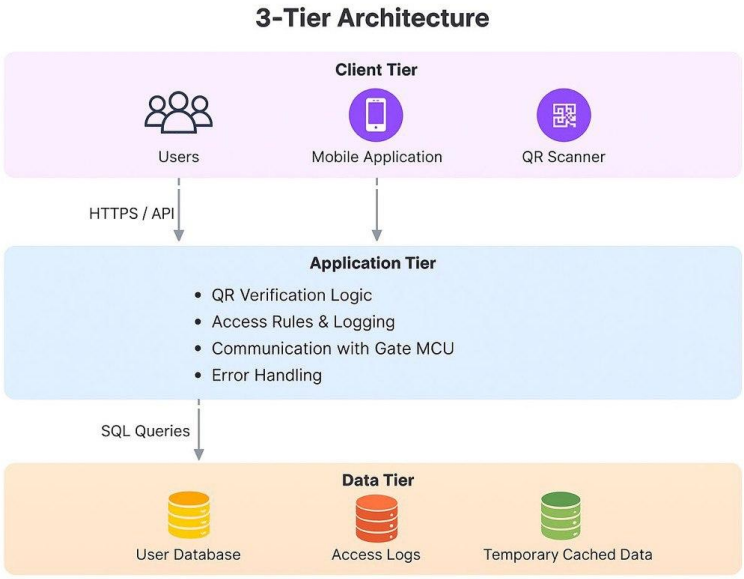


Figure 35. Alternative architecture [2]

5.5.1.3 Third Alternative

The third alternative design for this System uses the Hybrid 3-Tier & Layered architecture.

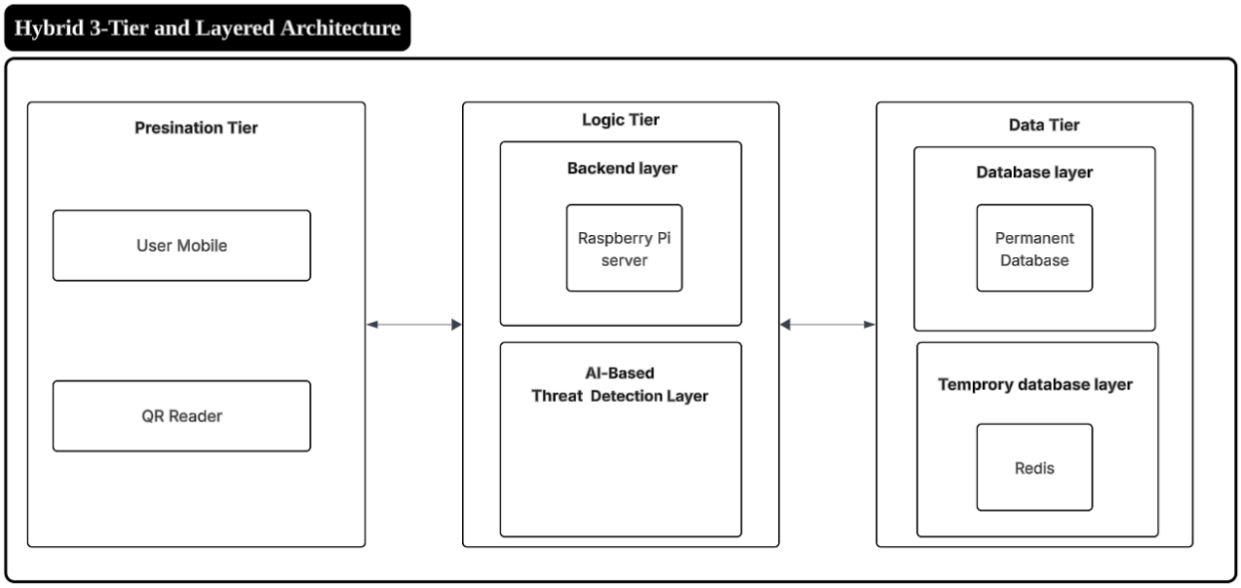


Figure 36. Alternative architecture [3]

5.5.2 Select the Best Architecture Alternative

In this part we have depended on a special decision tree analysis known as **Kepner-Tregoe Decision Analysis** [7] to evaluate architecture and select the best architecture alternative. Our main object is to assess the architecture representing different system designs based on how well they meet the desired system requirements and constraints.

According to the journal article, the **Kepner-Tregoe Decision Analysis** [7] is as follows:

1. State the decision.
2. Develop objectives.
3. Classify objectives into MUSTs and WANTs.
4. Weigh the WANTs.
5. Screen alternatives through the MUSTs.
6. Compare alternatives against the WANTs.

1) State the decision

Which of the 3 proposed architectures (Event-Driven Microservices, 3-Tier and the Hybrid 3-Tier & Layered) best satisfies our **must-have** non-functional requirements (Maintainability and Usability) and scores highest on our criteria (Availability, Reliability, Performance, Security, Accuracy, Scalability, Interoperability, Portability).

2) Develop Objectives

Quality Attribute	**Source**
Availability	NFR-01
Reliability	NFR-02
Scalability	NFR-03
Performance	NFR-04
Security	NFR-05
Accuracy	NFR-07
Interoperability	NFR-08
Portability	NFR-10

3) Classify Objectives into MUSTs and WANTs

Category	**Objectives**
MUSTs	Maintainability and Usability
WANTs	Availability, Reliability, Performance, Security, Accuracy, Scalability, Interoperability, Portability

4) Weigh the WANTs

We will assume weights (sum = 1.0) reflecting importance:

WANT:	
Availability	0.16
Reliability	0.16
Performance	0.16
Security	0.18
Accuracy	0.14
Scalability	0.10
Interoperability	0.05
Portability	0.05

5) Screen the alternatives through the MUSTs

Note: During this step, we eliminated the Event-Driven Microservices Architecture before entering the rating matrix. Although the event-driven microservices architecture Figure (34) can satisfy several MUST-have quality attributes, it does not satisfy our project-level constrains. Since our team consists of only 2 members and we have only a single semester to complete the system, implementing a full microservices architecture, an event broker, its extra complexity and would not be feasible.

Additionally, the nature of our system does not require multiple domains or separate services; the system logic can be efficiently handled within a simpler architectural style. For these reasons, the event-driven microservices architecture was eliminated before the weighted evaluation stage.

6) Compare alternatives against the WANTs

Quality Attribute	**Weight**	**3-Tier Rate**	**3-Tier (Weight Rate)**	**Hybrid Rate**	**Hybrid (Weight Rate)**
Availability	0.16	4	0.64	5	0.80
Reliability	0.16	4	0.64	5	0.80
Performance	0.16	3	0.48	4	0.64
Security	0.18	3	0.54	5	0.90
Accuracy	0.14	3	0.42	5	0.70
Scalability	0.10	3	0.30	4	0.40
Interoperability	0.05	3	0.15	4	0.20
Portability	0.05	3	0.15	5	0.25
Total Score	**1.00**		3.32		4.69

Figure 37. QA Against Architecture Weights

As we can see from the above image, we first evaluated our quality attributes (5 being the best, and 1 being the worst). Using the scoring matrix, our team assigned ratings to both architectures based on how well each one satisfies the weighted criteria. The final weighted results are shown in Figure (37)

- **Total Weighted Score (3-Tier Architecture) = 3.32**
- **Total Weighted Score (Hybrid Architecture) = 4.69**

Since the **Hybrid 3-Tier & Layered Architecture** achieved the highest score, this architecture was selected as the best-fit design for our system.

Evaluation Meeting & Reasoning

The above table was made through discussion within the teammates.

This is the table containing our very brief analysis:

Quality Attribute	Why 3-Tier = this rating	Why Hybrid 3-Tier & Layered = this rating
Availability	Relies heavily on the connection to the User Database; if the Data Tier is unreachable, the system fails.	The **Temporary database layer (Redis)** allows the system to function and grant access even if the main Database is temporarily unreachable due to lack of internet.
Reliability	A failure in the monolithic Application Tier can crash the entire verification process.	Layering isolates faults; if the AI layer fails, the **Backend layer** can still process basic access requests without crashing the system.
Performance	Direct SQL queries for every scan can cause latency, potentially exceeding the 1-second target	Using **Redis** for caching allows for sub-millisecond data retrieval, ensuring the system easily meets the ≤ 1 second processing requirement[3].
Security	Relies on standard encryption and static access rules, which may miss sophisticated attacks.	The dedicated **AI-Based Threat Detection Layer** proactively identifies anomalies, offering superior protection for sensitive user data
Accuracy	Uses static logic which may fail to detect complex abnormal access behaviors.	The architecture integrates a specific **AI model** capable of achieving the required 95% accuracy in identifying abnormal behavior
Scalability	Scaling requires replicating the entire Application Tier, which is resource-intensive.	The **Data Tier** and **Logic Tier** can be scaled independently, allowing the system to handle high loads across multiple gates efficiently.
Interoperability	High coupling in the Application Tier makes it difficult to	Standardized REST APIs between the **Presentation, Logic, and Data tiers** ensure

	integrate new hardware or services.	seamless communication with diverse hardware and external systems
Portability	The monolithic backend structure limits flexibility when moving to different server environments.	The modular design allows the web application to run consistently across different browsers and devices without code changes

Table 11. Reasoning we made in the evaluation meeting

5.6 Detailed Design of Selected Architecture

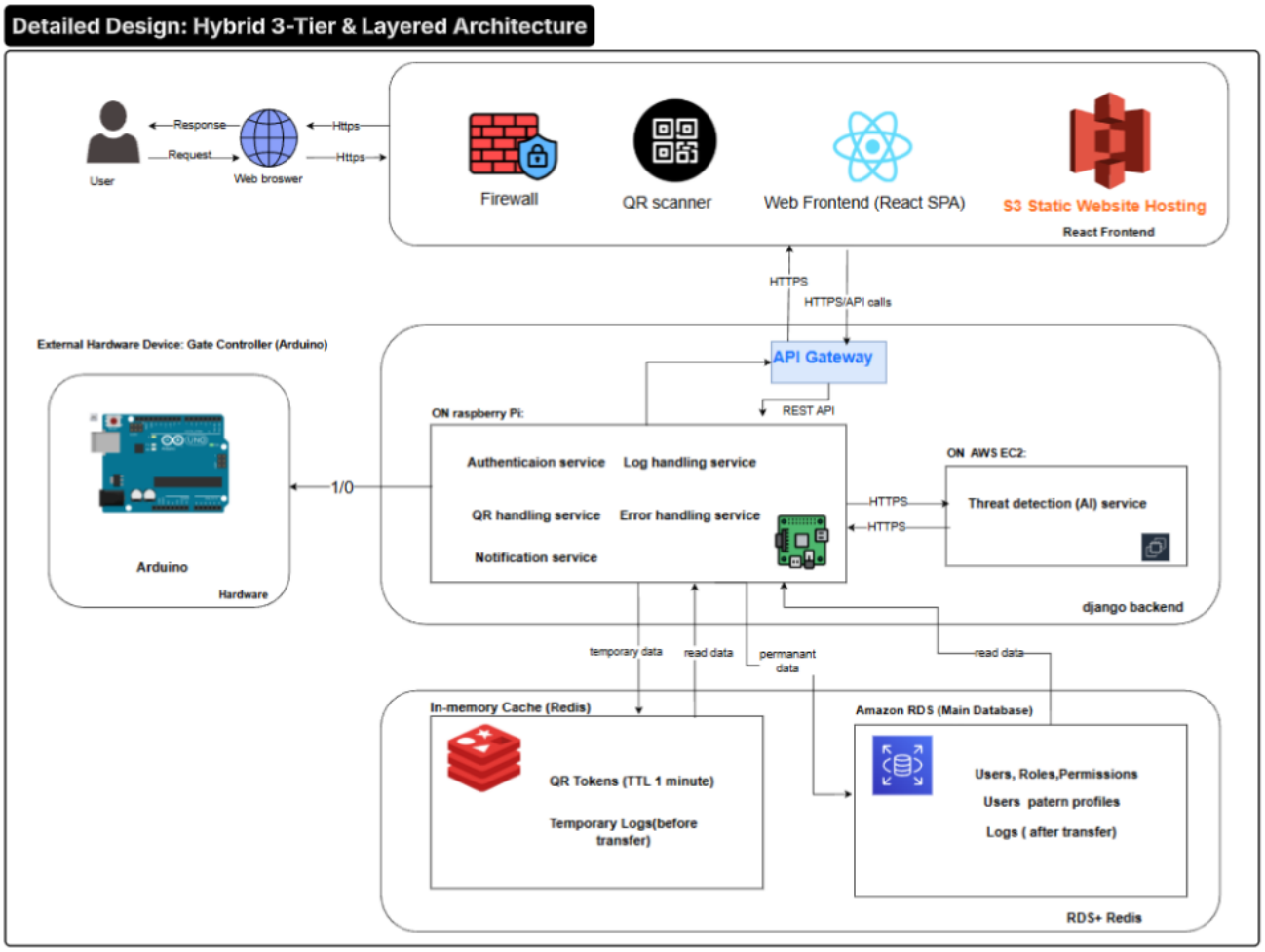


Figure 38. Detailed system architecture showing the Hybrid 3-Tier and Layered Design for the Solar-Powered Smart Entry System

5.6.1 Detailed Architecture Description

Component	Framework / Technology	Usage
Web Frontend	React (SPA)	User interface for login, QR request, notifications
Static Hosting	AWS S3	Hosts the frontend as a static website
API Gateway	Custom REST API Gateway	Routes requests between frontend, Raspberry Pi, and EC2 backend
Firewall	Network Firewall	Protects system from unauthorized access
QR Scanner	Hardware Scanner	Reads QR codes at the gate
Raspberry Pi	Raspberry Pi OS, Python Services	Hosts local microservices (Auth, QR, Logs, Error, Notification)
Authentication Service	Django (Python)	Validates users and communicates with RDS
QR Handling Service	Django (Python)	Generates and validates QR tokens
Notification Service	Django (Python)	Sends real-time notifications
Log Handling Service	Django (Python)	Processes local logs before storing in the cloud
Error Handling Service	Django (Python)	Captures and reports system errors
Arduino Gate Controller	Arduino IDE	Controls motor open/close
In-Memory Cache	Redis	Stores temporary QR tokens and temporary logs
Main Database	AWS RDS (PostgreSQL)	Stores users, roles, permissions, logs, and patterns
AI Threat Detection Server	AWS EC2	Runs the AI model for detecting abnormal access behavior

Table 12. Detailed Architecture Description

5.7 Simulation of Backend Functions Using Django

In this part of the document, we present our simulation of the QR generation and validation component of our project. Although this simulation is not a formal requirement for Software Engineering Capstone I, we implemented it to remain aligned with the Electrical Engineering team, whose work includes a simulation component.

For this small prototype, we used Python with the Django framework as our backend. Postman was used to simulate the frontend environment for calling and testing our backend functions. (A complete frontend—such as React—may be developed later.)

We opened VS Code and configured our Django views as shown in Figure (40).

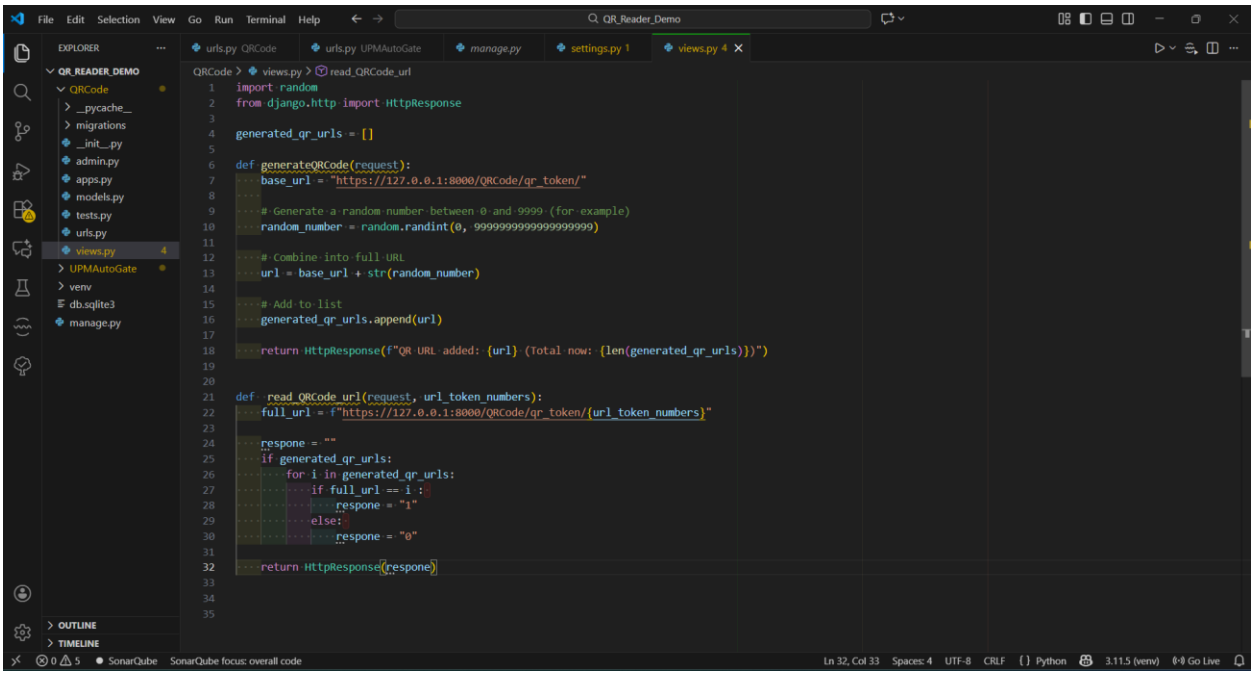


Figure 39. Django view functions used to simulate QR code generation and validation in VS Code.

These are our functions, each serving a specific purpose. The first function, **generate_qr_code**, is responsible for generating the QR code URL. The generated QR is stored in a small list, which in the real application would act as a buffer or a temporary database.

The second function, **read_qr_url**, is used to validate the QR code (represented as a URL in this prototype). It checks whether the QR code is valid, not corrupted, and not discarded.

Now we will demonstrate how the frontend is intended to interact with our backend simulation using Postman.

This URL represents the “Request QR Code” button. When we press **Send**, a QR code URL is generated through the function explained earlier. In later development stages, this URL will be converted into an actual QR image to be scanned by the user.

The returned result shows a small token which, in the real application, is intended to be an encrypted One-Time Token (OTT) that securely includes the user ID, expiration time, and timestamp. As shown in figure (40).

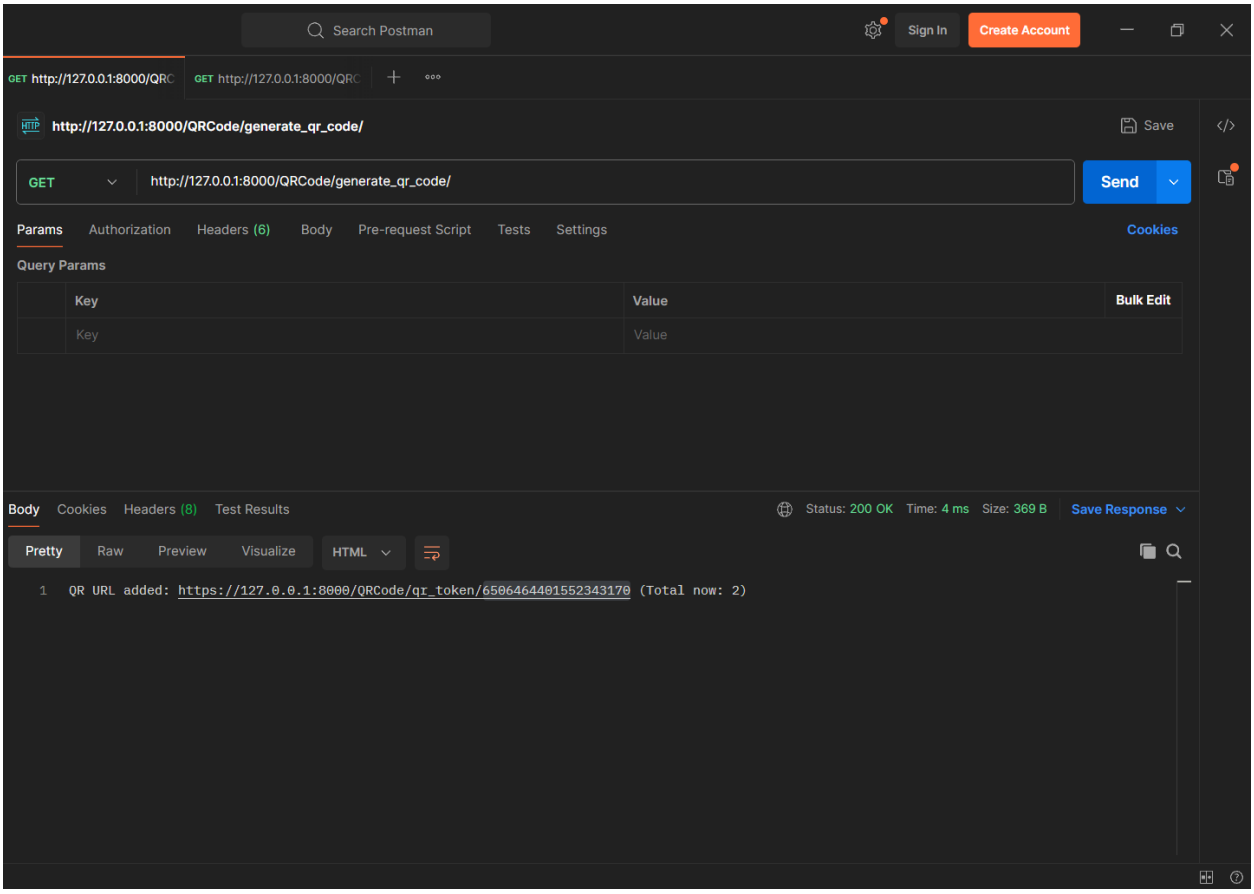


Figure 40. Postman request demonstrating the successful generation of a QR URL using the Django backend.

Now we will validate the QR code URL. This represents the scanning of the QR code. The token entered in this field simulates the QR code being scanned and sent to the backend. The backend then validates the token by checking whether it exists in our buffer or temporary database and ensuring that it has not been corrupted. As shown in figure (42).

Since this is the first time the QR code is used and it is valid the system returns **1**, indicating successful validation. In the real implementation, this signal would be received by the Arduino, which would then trigger the gate to open.

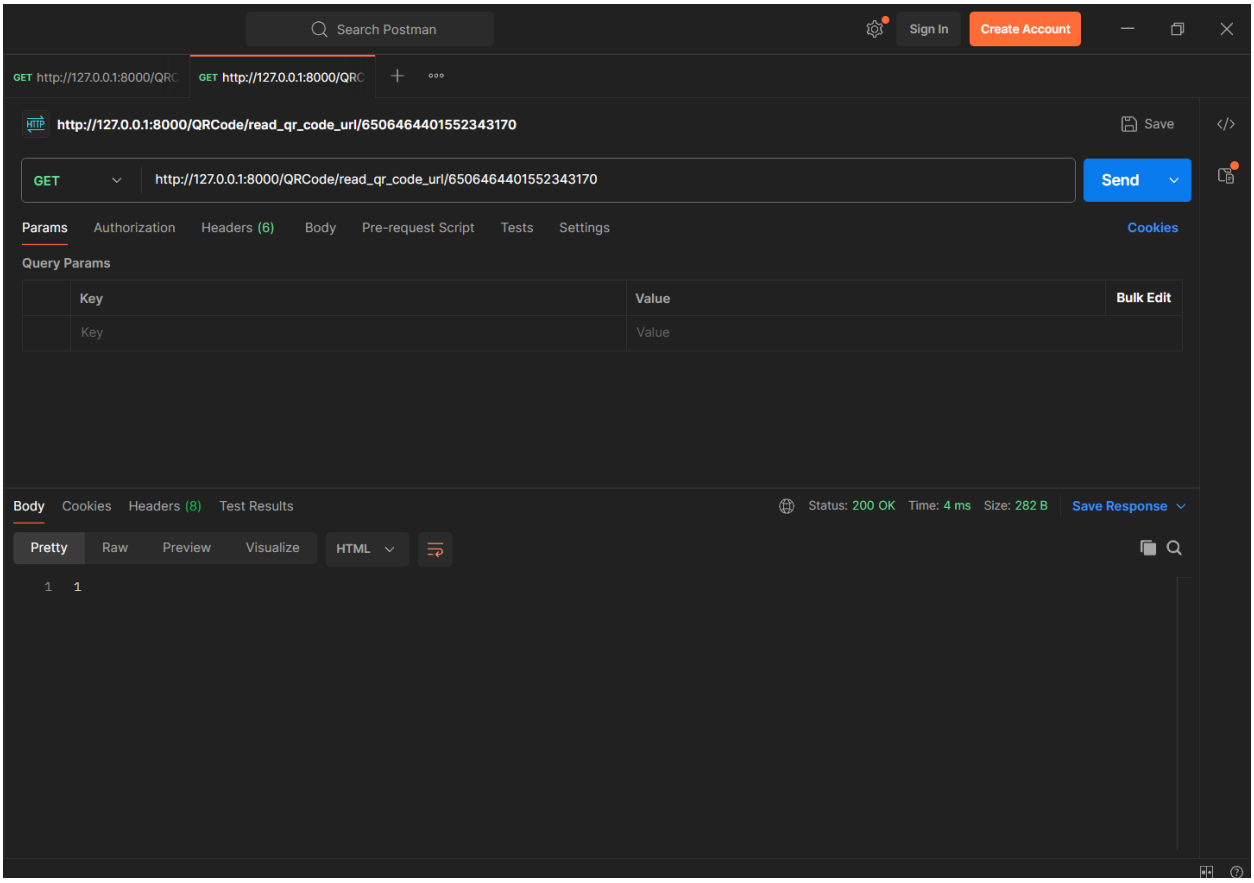


Figure 41. Postman request showing successful validation of a previously generated QR token.

In this case, the validation result returns **0**, which indicates that the QR code is **invalid**. This happens when the token is either corrupted, expired, or no longer exists in our buffer or temporary database.

In the real implementation, receiving a **0** means the Arduino would **reject the request**, and the gate would not open, ensuring that unauthorized or tampered QR codes cannot be used.

As shown in Figure (42), the backend correctly identifies the invalid token and prevents access.

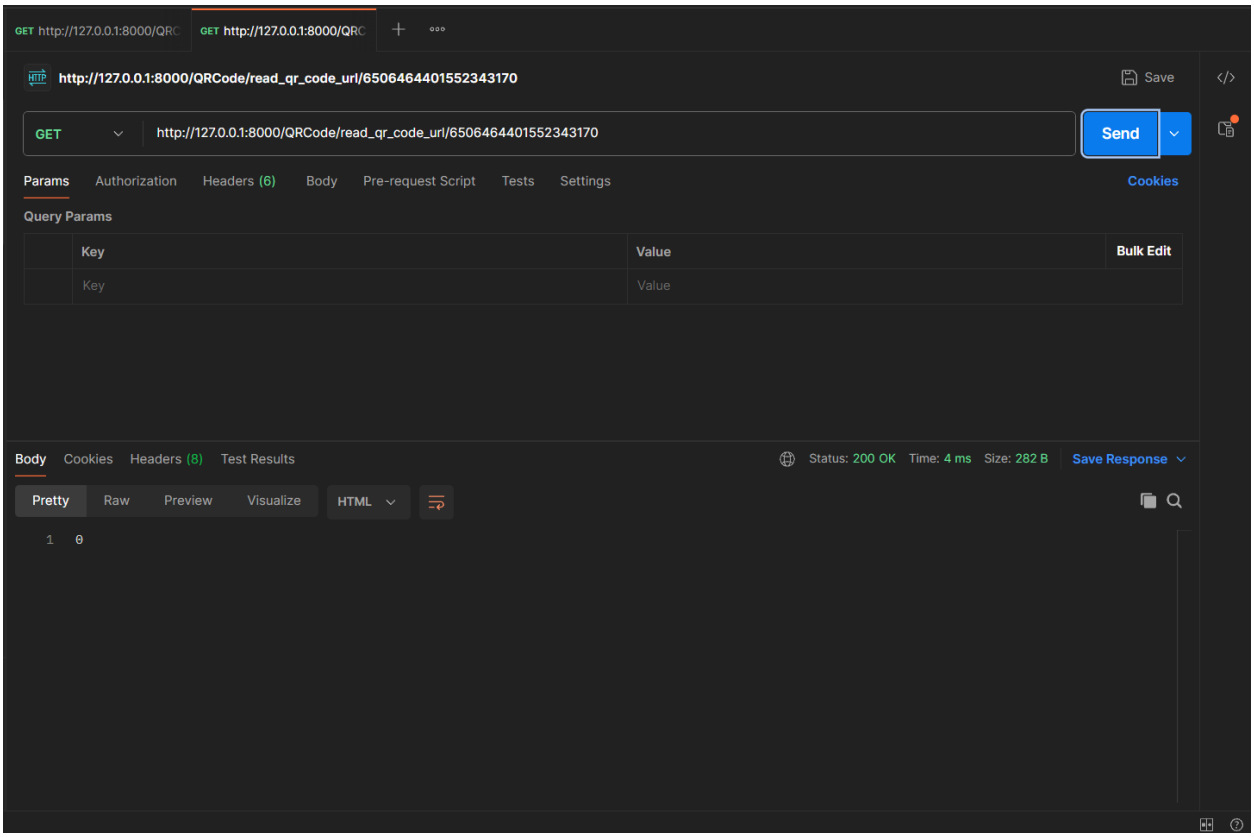


Figure 42. Postman request showing a failed QR validation attempt for a token not found in the system.

This simulation demonstrates that our tools are capable of achieving the intended goals and functionality of our application — and they perform exactly as required.

5.8 UI/UX Design

The user interface for the **Solar powered Smart Entry System with AI threat Detection.** application is designed for maximum clarity and ease of use. Key dashboards provide instant status updates, activity logs, and system analytics. The design prioritizes **security and accessibility**.

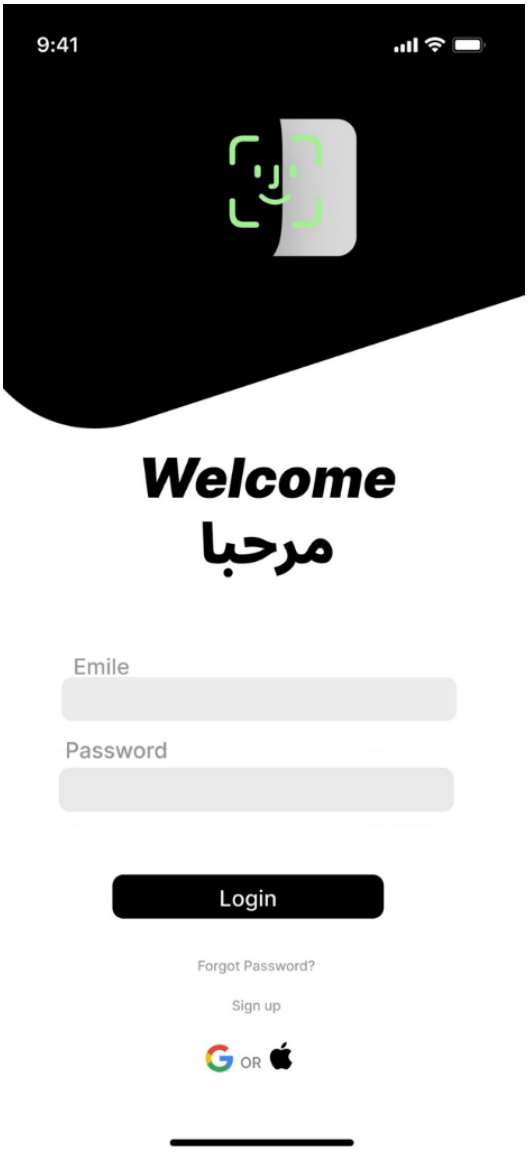


Figure 43. Figure Login/Welcome Screen

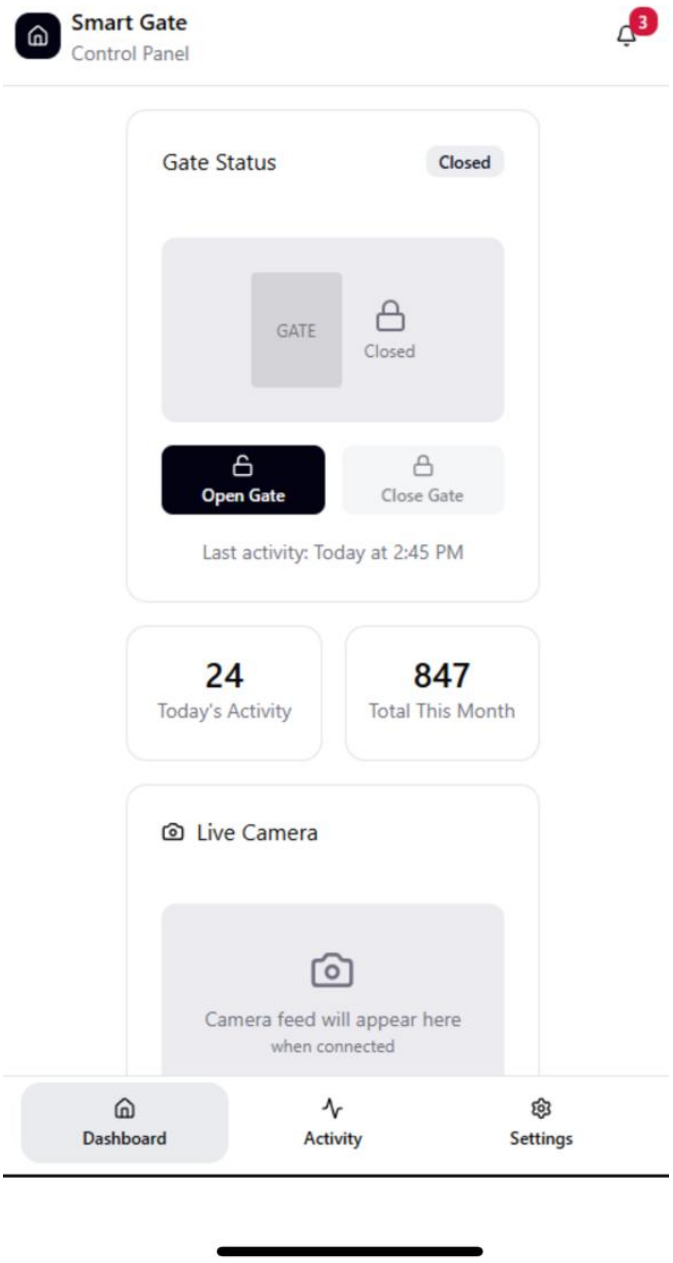


Figure 44. Figure Smart Gate Control panel

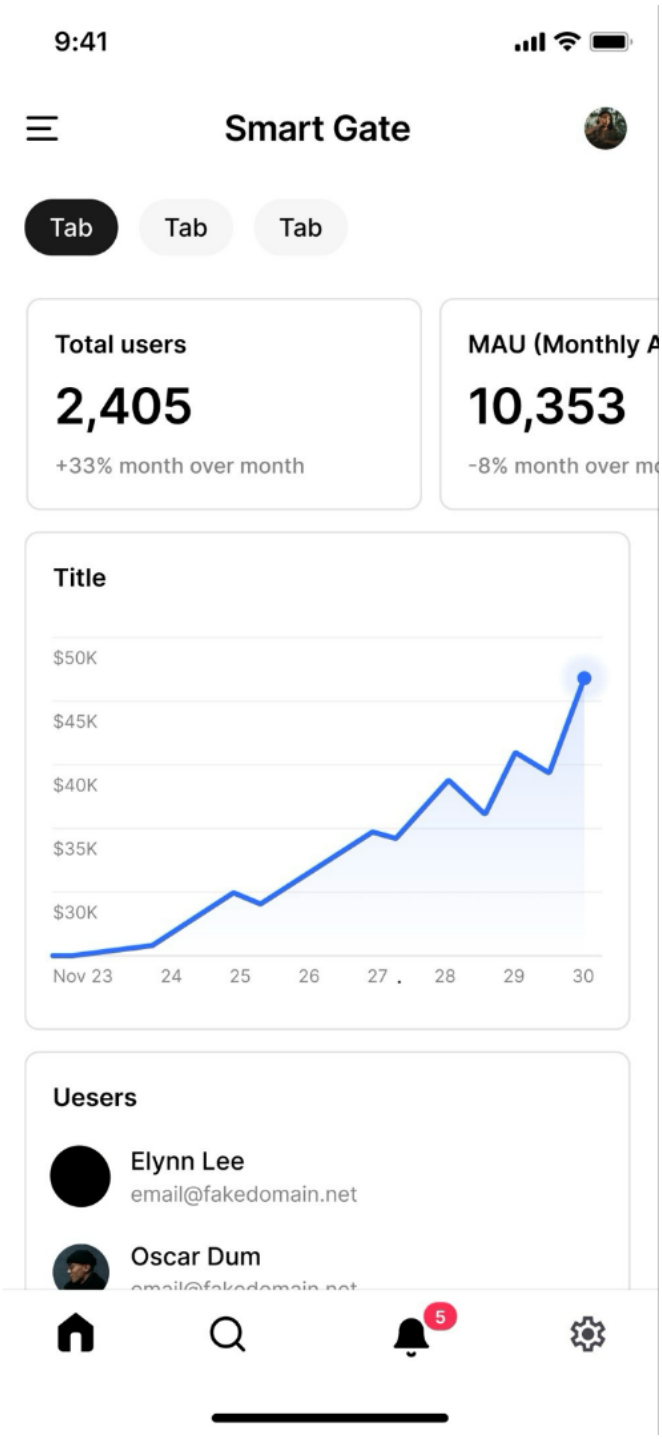


Figure 45. Figure Analytics/Admin Dashboard

CHAPTER 6: PROFESSIONAL, ETHICAL STANDARDS AND PROJECT IMPACT

This chapter explains the ethical standards we followed during the project and shows how our system creates positive impact.

6.1 Professional and Ethical Standards

In this part of the document, we highlight the key ethical principles from the Software Engineering Code of Ethics that guided the development of our System [8]. These principles ensure integrity, user protection, and responsible collaboration throughout the project.

Principle 2.05

We protect all confidential information obtained during the project, ensuring that access logs, user identities, and system configurations remain private and handled according to legal and ethical requirements.

Principle 4.01

We make all technical decisions with the goal of supporting human values, designing the system to enhance safety, convenience, and overall user experience for the university community.

Principle 7.01

We encourage all team members, across both SE and EE disciplines, to adhere to ethical practices, maintain professionalism, and support each other in delivering a reliable and responsible solution.

Principle 7.03

We fully credit the contributions of all team members and avoid taking undue credit, ensuring transparency and fairness in documenting work done by each contributor.

6.2 Project Impact

1. Saudi Vision 2030

Since our system uses solar power and AI-based threat detection, it directly supports Vision 2030 goals for Humanization Perspective, sustainability, digital transformation, smart campuses, and improved safety. It also reduces operational costs.

2. National Cybersecurity Authority (NCA) Policies

Since our system implements secure QR authentication, encrypted communication, and AI anomaly detection, it aligns with NCA policies such as Cloud Cybersecurity Controls by enhancing trust, digital safety, and secure access.

3. Renewable Energy Policy — Ministry of Energy

Since our gate operates on solar energy, the system supports Saudi renewable energy initiatives by reducing electricity consumption and promoting clean-energy adoption in campus infrastructure.

4. Environmental Sustainability Policies — NCEC

Since our system reduces energy consumption and operates with environment-friendly components, it aligns with national environmental compliance policies aimed at reducing carbon footprint and improving energy efficiency.

5. Digital Government Authority (DGA) — Digital Transformation Policies

Since our solution replaces physical cards with secure digital QR codes, and relies on APIs and real-time digital logs, it aligns with DGA principles of smart services, digital automation, and enhanced user experience.

CHAPTER 7: ELECTRICAL ENGINEERING TAKS

7.1 Overview

This chapter covers electrical engineering responsibilities in the project. The EE tasks focus on the design, simulation, and validation of the hardware system required to automate the gate operation. The work includes control logic design, motor operation verification, power considerations, and system visualization using simulations and 3D modeling. The objective is to ensure that the proposed system is electrically safe, feasible, and ready for physical implementation in the next semester.

7.2 Preliminary Design

The preliminary design describes the operation of the automated gate system, focusing on control logic, signal flow, and system behavior. At this stage the design is verified through simulations rather than physical hardware implementation.

7.2.1 Feedback Loop Design

- Setpoint /Reference: Desired gate position (from QR scan)
- Controller: Arduino compares setpoint with feedback, generates control signals
- Actuator: Relay + DC motor moves gate.
- Feedback: Sensor sends position back to controller.
- Output: the status of the gate.

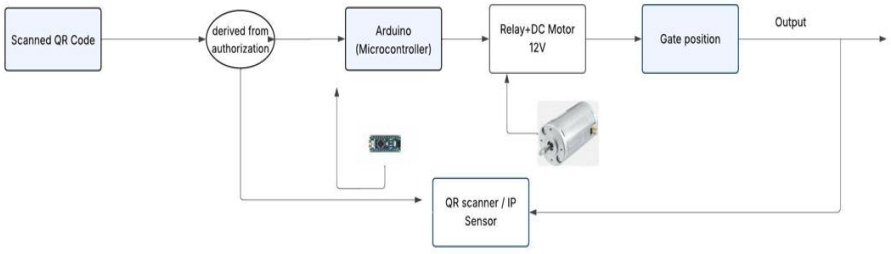


Figure 46. Feedback Loop design for the gate

7.2.2 Flowchart Design:

- Start: System powered on.
- Scan QR: Input device reads code.
- Validation: Arduino checks authorization
  - Valid: proceed to open the gate.
  - Invalid: system waits for next input.
- Gate closing: After a timed delay, the gate automatically closes.
- End/ Waiting state:
  - The system returns to the initial ready state, waiting for the next QR code scan.
  - The process then repeats for the next user

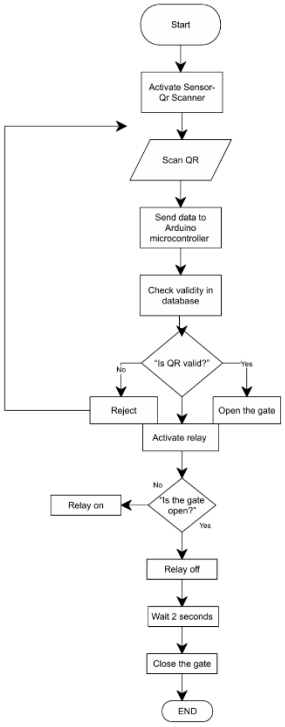


Figure 47. Flowchart design for the gate

7.3 Wiring Diagrams / Simulations

- Logic test simulation: Test Arduino logic, push-button inputs, and motor control via driver.
- Motor Feasibility Simulation: Test DC motor operation with SPST switches for direction control.

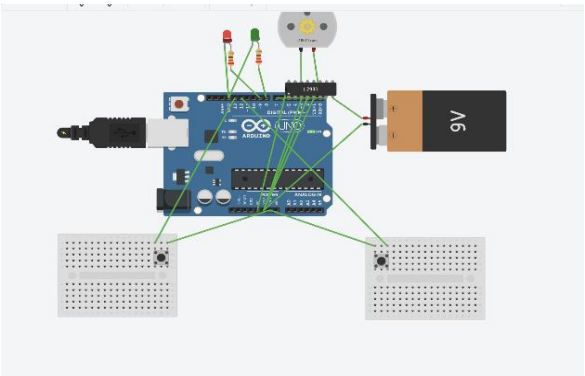


Figure 48. Pressed close-button switch

7.3 3D Layout

A 3D model was developed to illustrate the physical layout of the system components including the Arduino, DC motor, relay module, battery, and solar panel. The layout helps in understanding spatial arrangement, enclosure planning, and component placement prior to building the actual prototype.

A: top view

B: front view

C: bottom view

7.5 Specification, Limitation, and conditions

This subsection highlights system assumptions based on simulation-level design.

7.5.1 Specifications

-Arduino Uno (5V, max 40mA/pin).
-12V DC motor tested under 24V supply.
-LED + resistor simulating motor activation.
-Push buttons simulating QR input.
-Manual SPST switches used for the direction of the motor.

7.5.2 Limitations:

-Simulation restricts current output.
-QR scanner functionality is not included in the simulation.
-Solar panel charging behavior is not simulated.
-Environmental factors such as the temperature, light level, delays are not considered.

7.5.3 Restrictions and conditions of operation:

-Arduino must operate within safe current limits.
-Motor operation is simulated; no real machinal load is applied.
-Gate position feedback is assumed ideal.
-Real-world delays, friction, and inertia are not modeled.

7.6 Preliminary Results

Two simulations were conducted to verify the system logic and motor feasibility. Figures of the simulations are included to illustrate the results.

7.6.1 Logic test simulation

This simulation tested the Arduino control logic using push buttons to simulate QR code inputs and LEDs to indicate gate operation. The motor driver safely controlled direction, and the system correctly responded to forward and reverse commands, confirming proper logic operation.

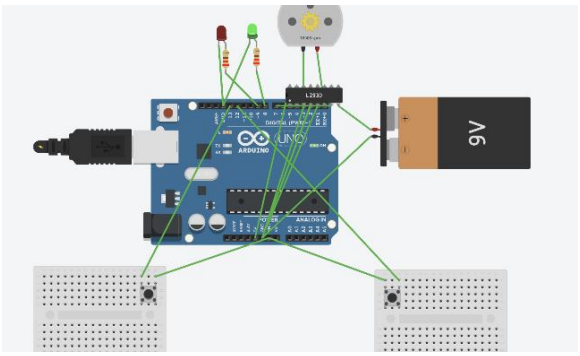


Figure 49.Pressed open-button switch

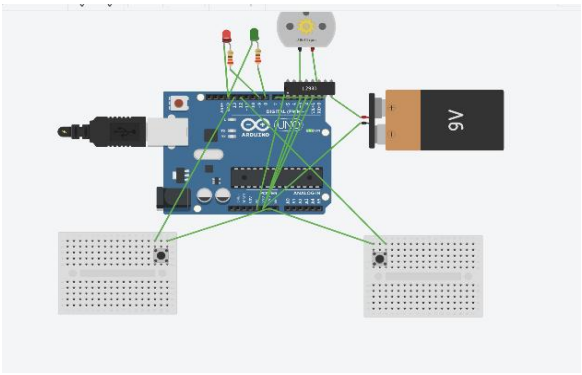


Figure 50. Pressed close-button switch

7.6.2 Motor Feasibility Simulation

This simulation verified the DC motor operation and direction control using SPST switches. The motor responded correctly to forward and reverse commands, confirming safe and functional operation before building the physical prototype.

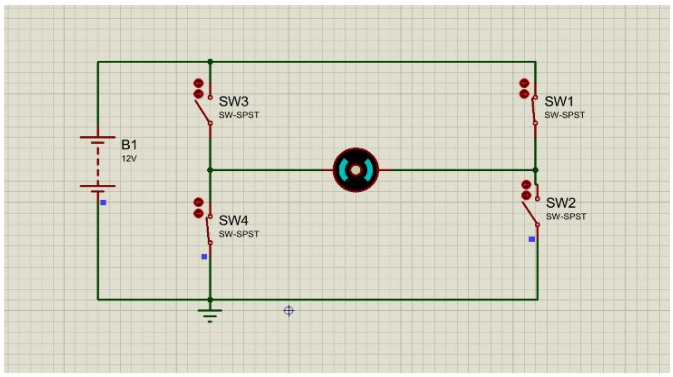


Figure 51. Clockwise direction (open status)

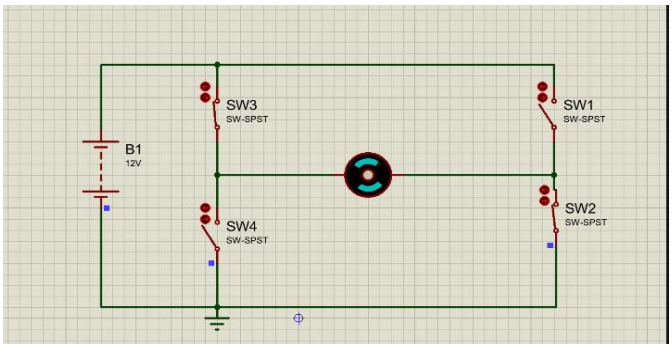


Figure 52.Counter clockwise direction (close status)

CHAPTER 8: NEW KNOWLEDGE

Throughout this multidisciplinary project, we gained significant new knowledge by working across both software and electrical engineering domains. During the requirements-gathering phase, our software team collaborated closely with the electrical engineering team to collect the hardware requirements. This experience allowed us to deepen our understanding beyond software and explore how hardware components operate, integrate, and influence system design.

When developing our UML diagrams—especially the Deployment Diagram—we took into consideration the physical architecture of the system. We met with the electrical team to clarify the hardware structure and the connections between both teams' components. This helped us view the system from a broader, systems-level perspective, rather than focusing solely on software. As a result, we learned to design solutions that align with real-world physical constraints and multi-team integration.

Additionally, we researched and selected the most suitable project methodology for both teams, ultimately choosing a Hybrid Waterfall–Agile approach. Applying this method was a new learning experience, balancing structured planning with flexible iteration.

Moreover, from the Electrical Engineering perspective, we also gained valuable insight into the Software Engineering lifecycle. We were able to learn and contribute effectively—particularly in the hardware requirements section, where we participated in reviewing, validating, and ensuring the accuracy and importance of all hardware-related requirements. We contributed by reviewing and validating hardware requirements during simulation, ensuring their correctness and relevance.

We also developed the Business Model Canvas for our project using the knowledge gained from the *Innovating with the Business Model Canvas* course (see Appendix H). This exercise enabled us to systematically identify customer segments, value propositions, revenue streams, and other key business elements, ensuring that the proposed Solar-Powered Smart Entry System with AI Threat Detection is supported by a coherent, viable, and sustainable business strategy.

CHAPTER 9: PROJECT IMPLEMNTAION, RESULTS, AND CONCLUSTION

9.1 Overview

This chapter presents the project plan, results from preliminary simulations, discussions of findings, and future work recommendations. It also outlines the estimated budget, roadmap, and training activities, followed by the conclusion drawn from project.

9.2 Results, Discussion, and Future Work

The preliminary simulations that were conducted successfully validated the control logic and operational sequences of the system. The Arduino correctly processed inputs from push buttons to test manual gate control, while the second simulation tested the 12V DC motor operation with SPST switches. Both simulations demonstrate that system logic and hardware selection are feasible for building a functional prototype.

The design choices and simulation results are supported by previous studies. Reference [4] highlights the importance of real-time logging and automated threat detection for secure smart systems, which informs the security and monitoring aspects of our design. Reference [13] presents an intelligent Automated Gate System using QR codes, emphasizing both hardware-software integration and the need to overcome QR detection challenges, such as lighting and connectivity. Our project aims to improve this by implementing more reliable detection, supporting offline verification. Reference [9] introduces a Smart Campus Gate System that reduces waiting time and simplifies operations. Building on the concept, our system focuses on faster, more accurate gate operation using efficient hardware-software and improved control logic.

In the next semester, the project will progress from simulation to prototype implementation. The system will be tested under realistic conditions, including varying light levels and different QR codes, to evaluate accuracy and reliability. Ensuring that project develops from validated simulation to functional and reliable prototype.

9.3 Conclusion

This project proposes a solar-powered automatic gate system integrated with QR code-based access control to improve efficiency and security in campus or organizational entrances. The preliminary simulations conducted in Tinkercad and Proteus validated the control logic, gate operation sequences, and the basic hardware functionality, demonstrating that the selected components are suitable for the system. The results also confirm that the approach is aligned with previous studies emphasizing automation, efficient access control, and system security. Future recommendations include implementing the physical prototype with real QR scanners, relay, motor, and solar power, testing under realistic conditions, enhancing QR code detecting accuracy, adding notification features, and optimizing energy usage to ensure fully functional, reliable, and efficient automatic gate system.

REFERENCES

[1] N. Jamous, G. Garttan, D. Staegemann, M. Volk, and H. Management, "Methods Efficiency in IT Projects," vol. 1, 2021, Available: https://scholar.archive.org/work/ij26t2bvozeztjmruur2hh3p4e/access/wayback/https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1233&context=amcis2021

[2] N. Koceska and S. Koceski, "Hybrid project management as a new form of project management," *Journal of Applied Economics and Business*, vol. 10, no. 4, pp. 16–23, Dec. 2022, Available: https://eprints.ugd.edu.mk/31403/

[3] "Log Management as an Enabler for Data Protection and Automated Threat Detection," *ISACA*, 2019. https://www.isaca.org/resources/isaca-journal/issues/2023/volume-4/log-management-as-an-enabler-for-data-protection-and-automated-threat-detection

[4] N. Hiremani *et al.*, "Artificial Intelligence-Powered Contactless Face Recognition Technique for Internet of Things Access for Smart Mobility," *Wireless Communications and Mobile Computing*, vol. 2022, pp. 1–11, Sep. 2022, doi: https://doi.org/10.1155/2022/8750840.

[5] "Systems and software engineering - Life cycle processes -Requirements engineering." Available: https://drkasbokar.com/wp-content/uploads/2024/09/29148-2018-ISOIECIEEE.pdf

[6] "About the Unified Modeling Language Specification Version 2.5.1," *www.omg.org*. https://www.omg.org/spec/UML/2.5.1/About-UML

[7] T. F. J. Oosthuizen, "The application of a selection of decision-making techniques by employees in a transport work environment in conjunction with their perceived decision-making success and practice," *Journal of Transport and Supply Chain Management*, vol. 8, no. 1, p. 9, Mar. 2014, doi: https://doi.org/10.4102/jtscm.v8i1.118.

[8] "(PDF) Software Engineering Code of Ethics and Professional Practice," *ResearchGate*. https://www.researchgate.net/publication/278417404_Software_Engineering_Code_of_Ethics_and_Professional_Practice

[9] “Guideline of Digital Project Management.” Accessed: Dec. 06, 2025. [Online]. Available: https://dga.gov.sa/sites/default/files/2024-08/Guideline%20of%20Digital%20Project%20Management%20-%20v1.0_0.pdf

[10] Alora Bopray, “The Best Solar Gate Openers - Today’s Homeowner,” *Today’s Homeowner*, May 22, 2025. https://todayshomeowner.com/solar/guides/solar-gate-openers/. (accessed Dec. 08, 2025).

[11] M. Garwood, *Maintaining a Stable Electricity Grid in the Energy Transition*, ICSC-CIAB Report, Jan. 31, 2024.

[12] M. Rahman, et al., “Smart Access Control System Using QR Code,” International Journal of Advanced Computer Science and Applications (IJACSA), vol. 13, no. 5, pp. 512–518, 2022. [Online]. Available: https://thesai.org/Publications/ViewPaper?Volume=13&Issue=5&Code=IJACSA&SerialNo=82

[13] A. Singh, et al., “Design and Implementation of Smart Gate System Using QR Code,” International Research Journal of Engineering and Technology (IRJET), vol. 8, no. 6, pp. 245–250, 2021. [Online]. Available: https://www.irjet.net/archives/V8/i6/IRJET-V8I6165.pdf

[14] N. Kumar, et al., “QR Code Based Secure Door Access System,” International Journal of Engineering Research & Technology (IJERT), vol. 9, no. 9, pp. 83–87, 2020. [Online]. Available: https://www.ijert.org/research/qr-code-based-secure-door-access-system-IJERTV9IS090083.pdf

[15] S. Bansal, et al., “Smart Campus Gate System Using QR Code,” IEEE Conference Paper, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10133127

[16] N. Ikpeze, E. C. Uwaezuoke, B.-M. Samiat, and K. M. Kareem, “Design and Construction of an Automatic Gate,” International Journal of Research and Publications, vol. 2, no. 2, pp. 123–131, Oct. 2021.

[17] R. Singh, R. Tirokar, P. Jagle, C. Vyas, N. Joshi, and R. K. Solanki, “Implementation of RFID-Based Security and Access Control System,” Proceedings of Sandip University Research Symposium, Nashik, India, 2021.

[18] N. Dileep Kumar and S. Shanthi, “Automatic Gate System Using Facial Recognition Technology,” International Journal of Computer Applications, vol. XX, no. YY, pp. 1–6, 2021.

[19] E. Hamid, C. G. Lim, N. Bahaman, S. Anawar, Z. Ayob and A. A. Malek, “Implementation of Intelligent Automated Gate System with QR Code,” International Journal of Advanced Computer Science and Applications (IJACSA), vol. 9, no. 10, pp. 359-363, Oct. 2018.

[20] Q. Team, “QR Code Gate Automation System: Complete Implementation Guide,” QRTRAC, Jan. 20, 2025. https://qrtrac.com/blogs/qr-code-gate-automation-system/ (accessed Dec. 14, 2025).

[21] “Smart Access | QR Code Access Control | NineSmart,” NineSmart, Oct. 17, 2023. https://ninesmart.io/smart-access/ (accessed Dec. 14, 2025).

APPENDIX A – GITHUB REPOSITORY LINK

GitHub Repository Link

APPENDIX B – JIRA PROJECT MANAGEMENT LINK

Jira Project Management Link

APPENDIX C – QUESTIONNAIRE FORM

Respondent: Ahsan Rahman		
Respondent Position: Associate Professor, Head of the Electrical Engineering Department, College of Engineering		
Contact Information: a.rahman@upm.edu.sa		
Company: University of Prince Mugrin		
Prepared By: Rama Mohamad		
Aspects	Questions	Answers
Demographic Information	1. How often do you have mobile internet or campus Wi-Fi available when arriving at the gate?	Availability of mobile internet/campus Wi-Fi: Usually available most of the time, though sometimes connectivity may be intermittent near the gate. However, mobile internet is always available.
	2. Do you prefer using your smartphone for gate access?	Smartphone for gate access: Yes, it is fine.
	3. Would you be comfortable showing a QR code from your phone to a gate scanner while in your car?	Showing QR code from car: Yes, I would be comfortable doing so.
Problem Identification	4. At what times do you usually face delays?	Times of delays: Currently, the system is quite efficient, but it would be even more interesting to automate it and reduce human involvement, making the security guards' jobs easier.
	5. Have you ever seen security opens the gates without checking identities?	Security opening gates without checking identities: I am not sure, but I think rarely.
	6. Do you think the gate system should work even if the guard is busy or not at the gate?	Gate system operation without guard: Yes, it should function independently if the guard is busy or absent.
	7. Do you feel that incidents at the gate are properly logged or traceable?	Logging of incidents: I am not sure, but it would be reassuring if all incidents were properly logged and traceable.

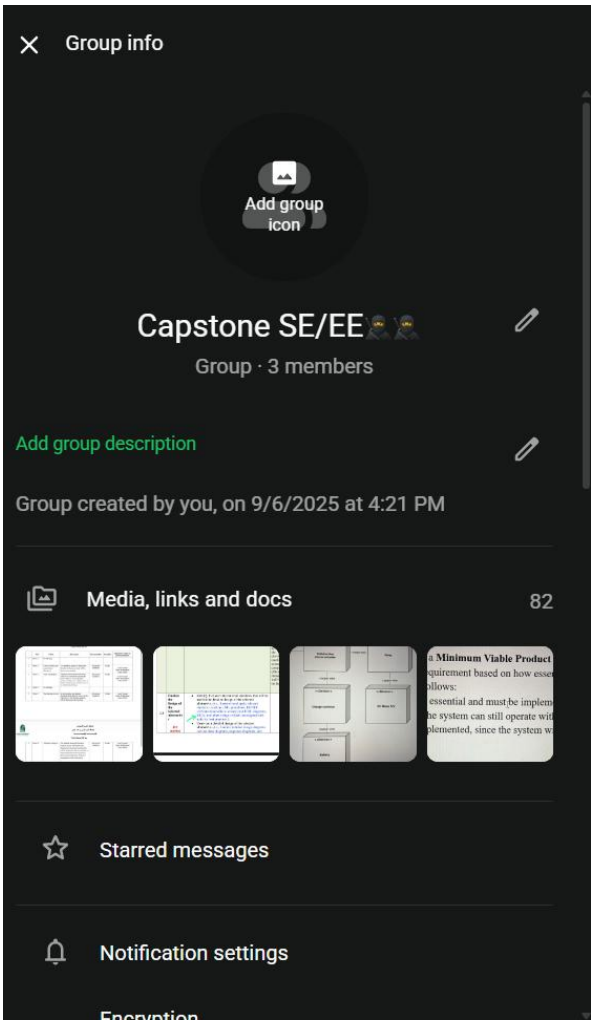
Requirements Gathering	8. Do you want to receive notification when your gate access is granted or denied?	Notification for access granted/denied: Yes, receiving notifications would be helpful.
	9. Would you like to have a dedicated page where they can view your own access history and previous gate activity?	Dedicated access history page: Yes, having a personal access history page would be useful.
	10. How long should the QR code remain valid? Do you consider one minute an appropriate duration before it is automatically deleted to prevent misuse?	QR code validity duration: One minute seems appropriate to prevent misuse.
	11. Do you think it would be beneficial to include a button that allows users to call security when needed?	Call security button: Yes, a quick-access button to call security would be beneficial in emergencies.

Table 13. Stakeholder questionnaire response

APPENDIX D – TIMING MINUTES

Meeting Minutes.docx

APPENDIX E – CAPSTONE SE/EE WHATSAPP GROUP



APPENDIX F – SUMMARY OF RELATED STUDIES

Reference	Journal Title	Problem that the journal tries to resolve	Their research methodology	Their solution to solve their problem
[1] Jamous et al., 2021	*Methods Efficiency in IT Projects*	Inefficiency and high risk in IT projects caused by selecting inappropriate SDLC methodologies	Comparative analysis of SDLC methodologies (Waterfall, Agile, Hybrid) based on project constraints and outcomes	Identifies strengths and weaknesses of Waterfall and Agile models and recommends Hybrid methodologies for complex IT projects
[2] Koceska & Koceski, 2022	*Hybrid Project Management as a New Form of Project Management*	Limitations of using pure Waterfall or pure Agile models in projects with mixed or changing requirements	Literature review and conceptual framework analysis	Proposes Hybrid Project Management combining the structured planning of Waterfall with the flexibility of Agile
[3] ISACA, 2019	*Log Management as an Enabler for Data Protection and Automated Threat Detection*	Poor system visibility, weak monitoring, and delayed threat detection due to ineffective log management	Industry analysis and security best-practice review	Introduces centralized log management with real-time monitoring and automated threat detection
[4] Hiremani et al., 2022	*Artificial Intelligence-Powered Contactless Face Recognition Technique for IoT Access*	The need for secure, contactless, and intelligent access control systems	Experimental research using AI-based facial recognition integrated with IoT devices	Proposes an AI-powered face recognition access control system for enhanced security

Table 14. Summary of Related Studies

APPENDIX G – RESEARCH OBJECTIVES TABLE

Objectives	Research Questions	Research Methodology	Output
To analyze the limitations of automated and manual gate operations particularly in security, entry speed, and assess their overall impact on campus operations.	1.1 What systems are currently used in university campuses (manual and automated)?	Literature Review & Competitor Analysis	Questionnaire result
	1.2 What security limitations are associated with existing manual and automated gate systems?	Literature Review & Questionnaire &Walkthrough	Literature Review Result The gaps in the existing system
To design and develop an automated campus gate system that uses dynamic QR-based contactless authentication and a threat-detection algorithm, while enhancing security through access logging that enables real-time traceability.	2.1 What is the most suitable architectural design for developing a web-based automated campus gate system with dynamic QR authentication and threat detection?	Architecture Analysis	Architecture to be used.
	2.2 How can the system be analyzed and designed using the Hybrid development methodology for system?	System analysis and design	UML models and architecture diagram
	2.3 How can access logging and monitoring be implemented in system to support real-time traceability and enhance campus security?	Development	Web based app
	2.4 How can an AI-based threat detection mechanism identify		

	abnormal or unauthorized activities in real time?	Algorithms	
To evaluate the operation and security of the smart campus gate system that utilizes dynamic QR-based contactless authentication and AI-powered threat detection, ensuring secure access control and real-time logging under various access scenarios.	3.1 How reliable and secure is the system's real-time logging and traceability during normal and abnormal access events? 3.2 How does the system control access using dynamic QR-based authentication under different access scenarios?	Training data Use case	Threat detection accuracy results. Training and Testing Data System logs demonstrating real-time monitoring and traceability

Table 15. Research Objectives Table

APPENDIX H – COURSERA CERTIFICATION

Innovating with the Business Model Canvas